

Neuronale Zustandsbestimmung für eingebettete Batteriemanagementsysteme

Illustrierte Erzählfassung mit Modellgrundlagen und Formeln

Leitgedanke. Welche Information benötigt ein neuronales Modell, wie zuverlässig verarbeitet es diese Information und wie wird daraus eine kompakte, praktisch einsetzbare BMS-Implementierung? Diese drei Fragen verbinden Datenrepräsentation, Robustheit und Embedded-Optimierung zu einer gemeinsamen Arbeit.

Lesart. Die Erzählung folgt 27 Bildstationen mit drei vorgeschalteten Schwerpunktansichten. Die Abbildungsnummern sind durchgängig die Nummern der *Dissertation*. Die Bilder stammen aus der aktuellen Dissertation. Beim BMS-Anforderungsbild markieren zusätzliche Konturen den jeweiligen Schwerpunkt, ohne die Originaldatei zu verändern. Die Stationen sind noch keine fertigen Folien.

Abschnitt	Stationen	Zeit
BMS und experimentelle Grundlage	1 bis 3	3:30
Zeitliche Repräsentation mit einem MLP	4 bis 8	6:30
Rekurrente Modelle und Robustheitsbenchmark	9 bis 17	8:15
Ausführung auf dem Mikrocontroller	18 bis 19	1:45
Pruning, Quantisierung und gemessene Effekte	20 bis 26	6:45
Gesamtergebnis und Ausblick	27	2:15
Erzählung		29:00
Puffer		1:00

Roter Faden. Modell gestalten, Verhalten prüfen, Implementierung optimieren. Beim MLP liegt der Schwerpunkt auf Deployment und Modellkomplexität. Der Benchmark verbindet Performance und Hardware. Die Optimierung verbindet Deployment und Hardware mit Accuracy als Kontrollgröße. Die Bereiche bleiben miteinander verbunden.

Zeitliche Konsequenz. Die Zeitblöcke schließen die kurzen Schwerpunktwechsel ein. Pro Ergebnisbild wird eine Beobachtung erklärt, nicht jedes Teilpanel. Die Zeit ist ein Probenziel. Rauschvertiefung und vollständige Gate-Gleichungen stehen als Reserve am Ende. Bei einer zu langen Generalprobe lassen sich die Syntheseansichten 6.15 und 6.16 gemeinsam besprechen.

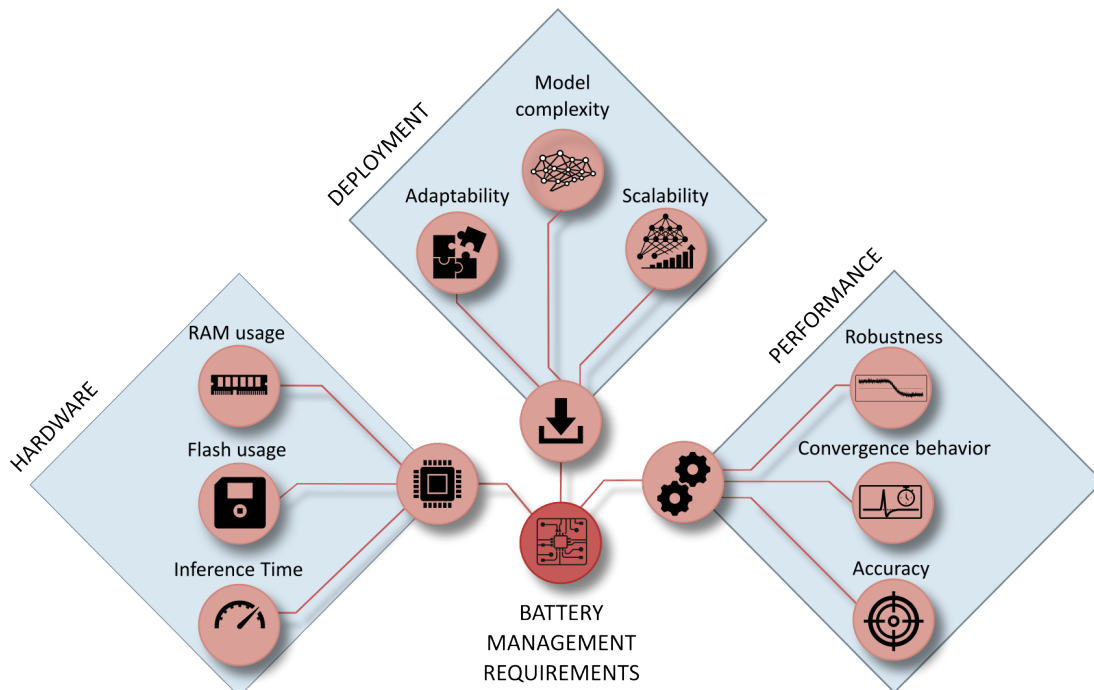
1 BMS-Anforderungen und Forschungsziel

1:15 min / Anforderungen

Wie lässt sich komplexes Batterieverhalten mit neuronalen Netzen beschreiben, die zugleich auf einem begrenzten Mikrocontroller betrieben werden können? Diese Frage steht im Zentrum meiner Arbeit. Ein BMS muss den Ladezustand und den Alterungszustand aus verfügbaren Messungen schätzen. Beide Größen beschreiben unterschiedliche Eigenschaften:

$$\text{SOC}(t) = \frac{Q_{\text{available}}(t)}{C_{\text{current}}(t)}, \quad \text{SOH}(t) = \frac{C_{\text{current}}(t)}{C_{\text{nominal}}}. \quad (1)$$

Eine gealterte Batterie kann voll geladen sein und trotzdem weniger Ladung speichern als eine neue. Gerade in dezentralen Speichern und mobilen Anwendungen muss diese Unterscheidung langfristig lokal verfügbar bleiben.



Dissertation, Abbildung 2.1. BMS-Anforderungen als gemeinsamer Bezugspunkt der Arbeit.

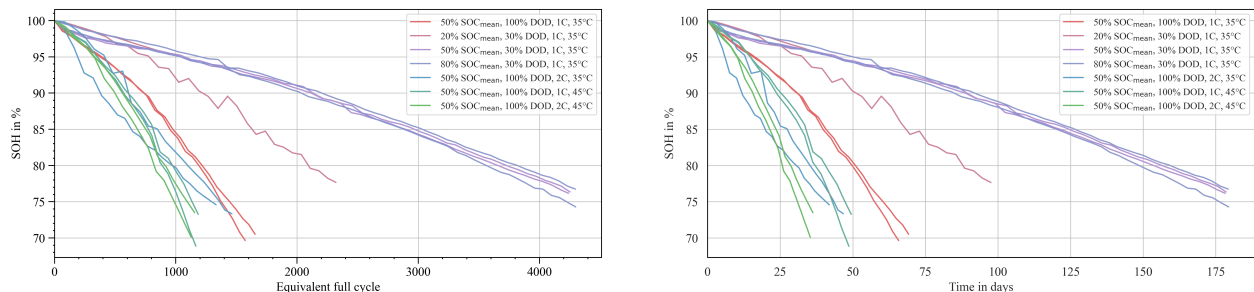
Deployment beschreibt die Gestaltung und Übertragung eines Modells in eine anpassbare BMS-Implementierung. Performance beschreibt die Güte seiner Zustandsbestimmung: Genauigkeit, Robustheit und Erholung. Hardware erfasst die dafür benötigten Ressourcen: Flash, RAM und Inferenzzeit. Modellkomplexität ist somit eine Entwurfsentscheidung, Speicher und Laufzeit sind ihre konkreten Auswirkungen.

Diese Anforderungen bleiben in der gesamten Arbeit verbunden. Zuerst gestalten wir ein Modell mit geeigneter zeitlicher Information. Danach prüfen wir sein Verhalten unter gestörten Bedingungen. Abschließend optimieren wir die Implementierung. Das Anforderungsbild zeigt jeweils, welcher dieser Aspekte gerade im Vordergrund steht.

2 Daten als Grundlage der Zustandsbestimmung

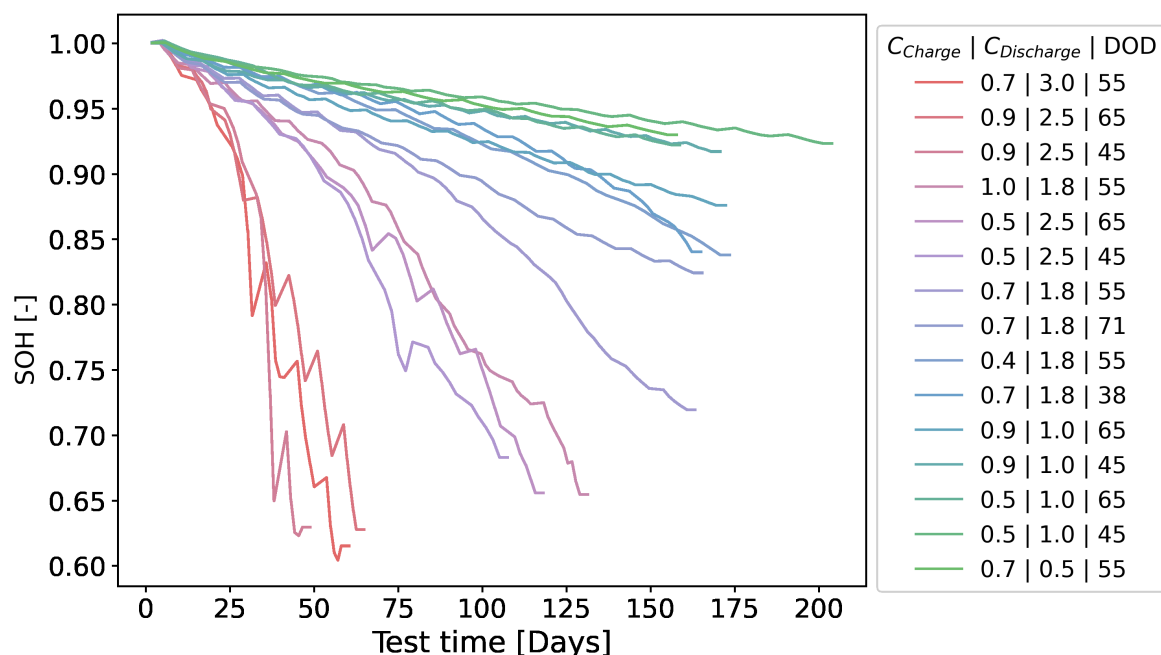
1:30 min / Datenbasis

Die Modelle benötigen Messreihen, in denen nicht nur einzelne Betriebszustände, sondern auch die Entwicklung der Batterie sichtbar wird. Die erste Untersuchung verwendet 14 NMC-Zellen unter unterschiedlichen Belastungsprofilen. Abbildung 4.1 zeigt die Alterung sowohl über Zyklen als auch über die verstrichene Zeit.



Dissertation, Abbildung 4.1. NMC-Alterungsverläufe in beiden ursprünglichen Darstellungen.

Für die folgenden Untersuchungen steht ein LFP-Datensatz mit 15 Zellen und acht Versuchsprofilen zur Verfügung. Die Profile variieren unter anderem Lade- und Entladerate sowie Entladetiefe. Das Design of Experiments verteilt die gewählten Bedingungen systematisch, damit nicht nur ein einzelner Betriebsfall beobachtet wird.



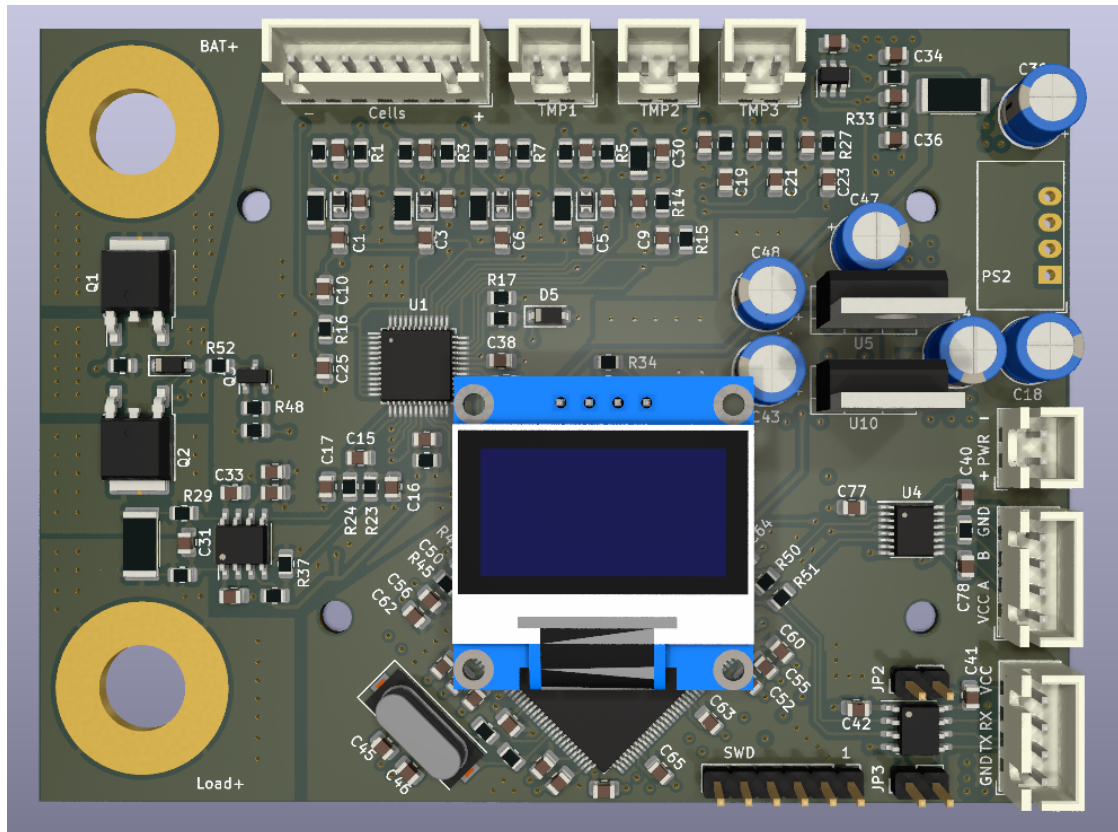
Dissertation, Abbildung 4.4. LFP-Zellen, Alterungsverläufe und unterschiedliche Betriebsbedingungen.

Diese Vielfalt ist wichtig, weil ein Modell Alterung von kurzfristigen Last- und Temperatureffekten unterscheiden muss. Für den Robustheitsbenchmark werden Trainings-, Validierungs- und Testzellen getrennt. Die sechs Testzellen prüfen somit die Übertragung auf andere Zellen, nicht nur auf spätere Messpunkte derselben Trainingsreihe.

3 Vom Messsystem zum programmierbaren BMS

0:45 min / Hardwarebasis

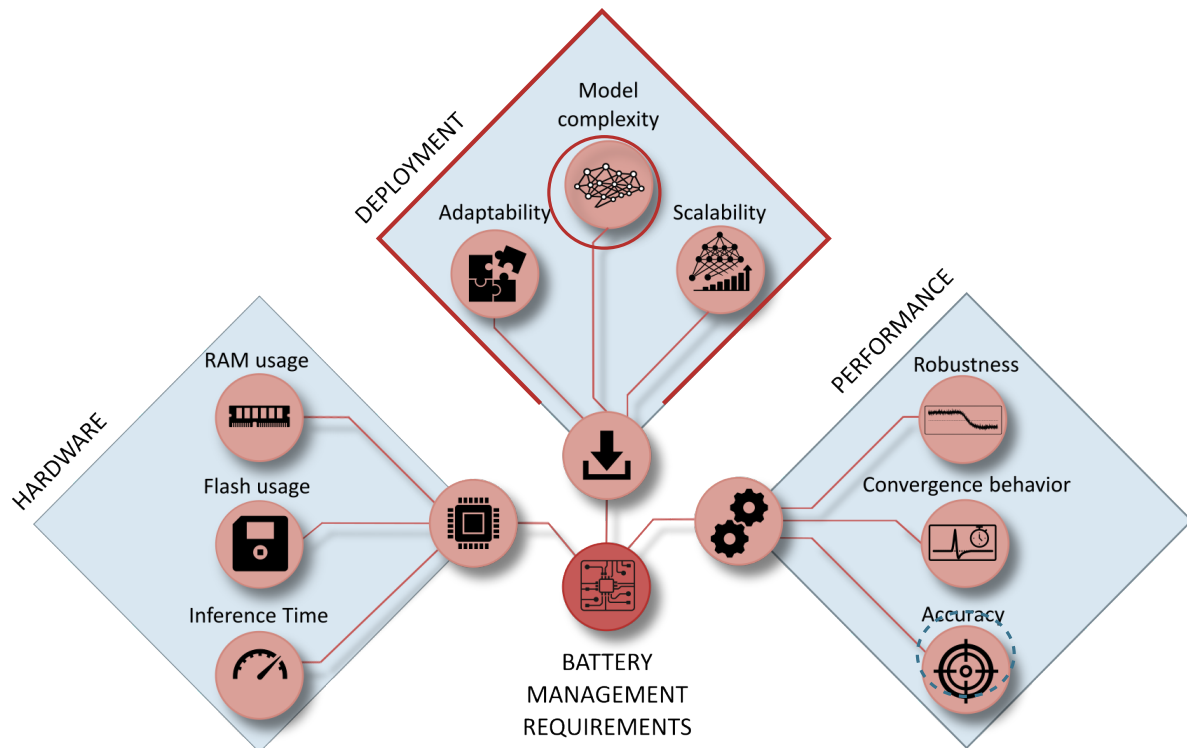
Ein trainiertes Netz ist noch kein Batteriemanagementsystem. Dazu gehören die Messung der Zellgrößen, die Überwachung und ein Controller, auf dem die Zustandsbestimmung ausgeführt werden kann. Die entwickelte Vier-Zell-Plattform verbindet einen Batterieüberwachungsbaustein mit einem programmierbaren STM32-Mikrocontroller.



Dissertation, Abbildung 4.6. Eigene BMS-Plattform mit Mess- und Verarbeitungselektronik.

Der TLE9012DQU übernimmt die zellnahe Überwachung. Der STM32H755 stellt die Umgebung für die eigene Software bereit. Die später gezeigten Laufzeit- und Speichermessungen wurden auf einem STM32H753 durchgeführt. Die eigene Platine und dieser Benchmark-Controller haben damit unterschiedliche Rollen. Gemeinsam machen sie deutlich, wo die Methoden eingesetzt werden sollen: zwischen realen Messsignalen und einer begrenzten lokalen Recheneinheit.

Zeitliche Repräsentation: Modellgestaltung



Dissertation, Abbildung 2.1. Schwerpunkt Deployment, insbesondere Modellkomplexität. Accuracy dient der Bewertung der gewählten Repräsentation. Rot markiert den Hauptschwerpunkt. Ein gestrichelter blauer Kreis markiert die ergänzend betrachtete Genauigkeit.

Der erste Teil setzt bei der Modellgestaltung an. Wie lässt sich zeitabhängiges Batterieverhalten bereits mit einer einfach aufgebauten neuronalen Architektur erfassen? Dazu untersuchen wir, welche Information wir dem Netz durch Merkmalsbildung und eine explizite Historie bereitstellen können. Die erreichbare Genauigkeit zeigt anschließend, ob diese Repräsentation funktioniert.

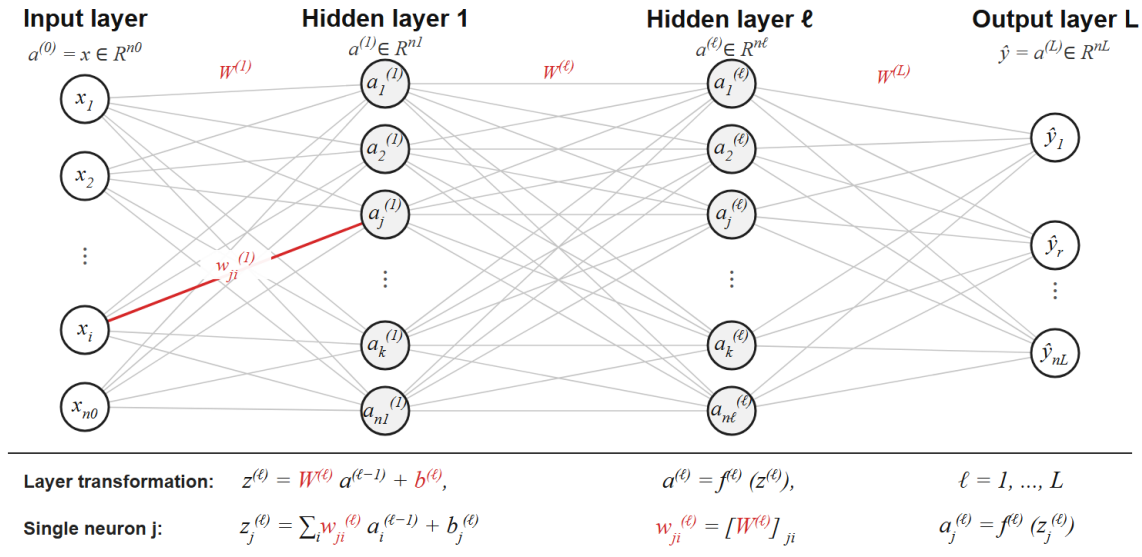
Im Anforderungsbild steht daher die Modellkomplexität im Vordergrund. Hardwareeffizienz ist das übergeordnete Entwurfsziel. Eine konkrete MCU-Vermessung folgt in den weiteren Teilen der Arbeit.

4 Ein MLP als Ausgangspunkt

1:30 min einschließlich Schwerpunktwechsel / Neuronale Grundlagen

Zunächst stellt sich die Frage, wie viel Modellstruktur überhaupt nötig ist. Ein Multilayer Perceptron ist ein geeigneter Ausgangspunkt, weil es aus einer Folge gut beschreibbarer Rechenoperationen besteht. Eine Schicht gewichtet ihre Eingänge, addiert einen Bias und wendet eine Aktivierungsfunktion an:

$$\mathbf{a}^{(\ell)} = f^{(\ell)}\left(W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad \text{ReLU}(z) = \max(0, z). \quad (2)$$



Dissertation, Abbildung 3.2. Eingänge, verborgene Schichten und Ausgabe eines MLP.

Die trainierbaren Parameter sind die Gewichte und Bias-Werte. Für eine voll verbundene Schicht mit n_{in} Eingängen und n_{out} Ausgängen gilt:

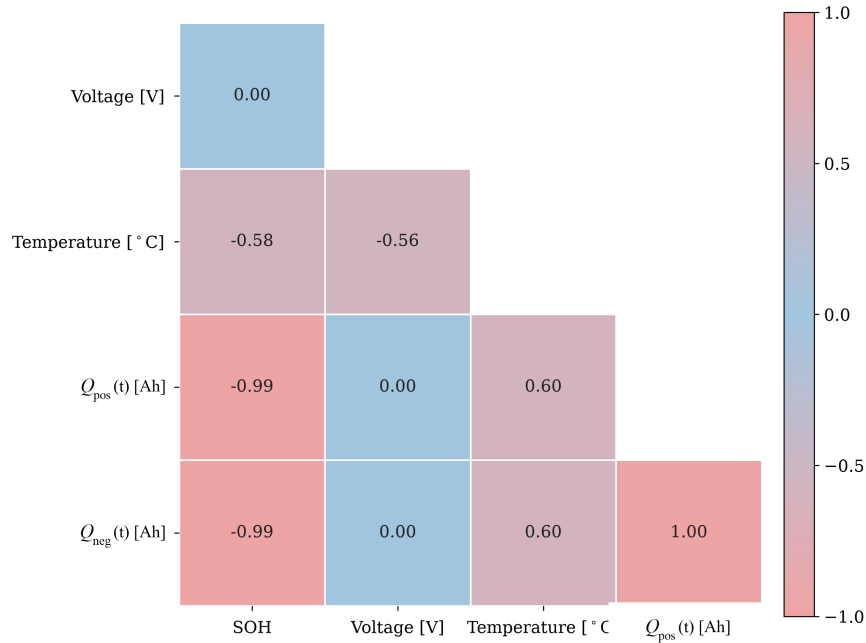
$$N_{\text{Parameter}} = n_{\text{in}}n_{\text{out}} + n_{\text{out}}. \quad (3)$$

Mehr Neuronen bedeuten damit nicht nur mehr Darstellungsfähigkeit, sondern auch mehr Speicher und Rechenaufwand. Ein MLP besitzt jedoch keinen internen zeitlichen Zustand. Werden nur aktuelle Messwerte eingegeben, steht die vorherige Belastung nicht automatisch zur Verfügung. Deshalb betrachten wir zunächst die Eingangsrepräsentation, bevor wir die Architektur aufwendiger machen.

5 Welche Information steckt in den Messgrößen?

1:00 min / Merkmalsbildung

Abbildung 5.1 liefert einen ersten Blick auf die Beziehungen im NMC-Datensatz. Die kumulierten Lade- und Entlademengen weisen eine starke negative Korrelation mit dem SOH auf. Für beide liegt der Korrelationskoeffizient ungefähr bei $-0,99$. Für Temperatur liegt er bei $-0,58$, für Spannung in dieser Auswertung nahe null.



Dissertation, Abbildung 5.1. Korrelationsmatrix der Messgrößen und abgeleiteten Merkmale.

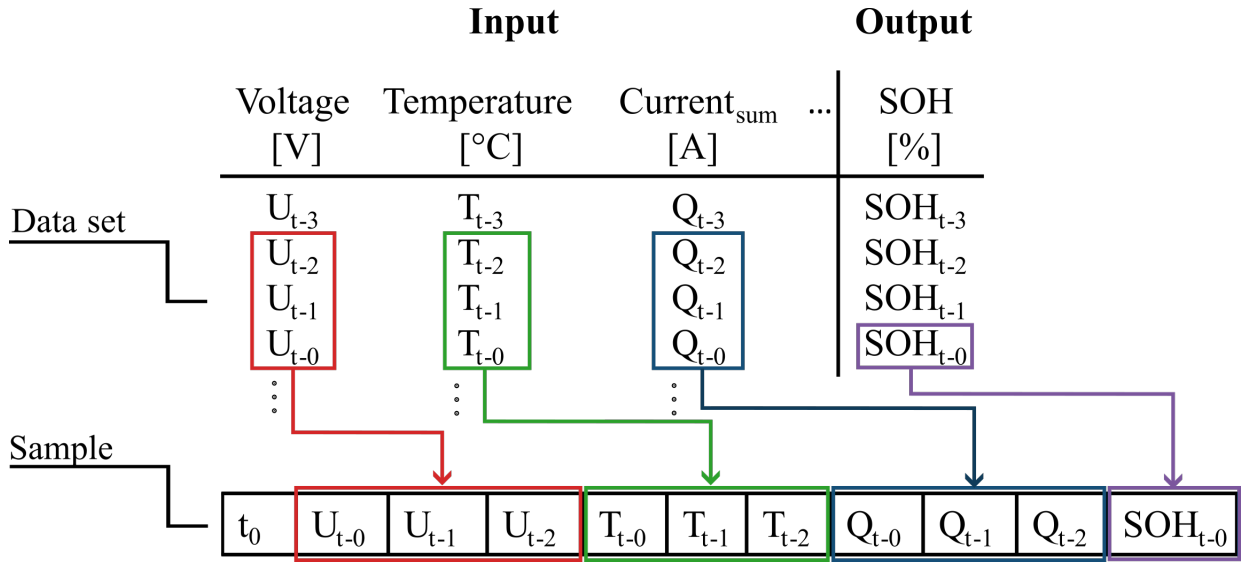
$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}. \quad (4)$$

Die Ladungsmengen enthalten bereits Belastungsgeschichte. Damit stellen sie dem Netz Information bereit, die im momentanen Stromwert allein fehlt. Diese Beobachtung motiviert Feature Engineering. Sie beweist allerdings weder eine kausale Wirkung noch die Wichtigkeit eines Eingangs im trainierten Netz. Insbesondere kann eine Größe mit geringer linearer Korrelation in Kombination mit anderen Eingängen dennoch nützlich sein.

6 Zeitlicher Kontext ohne rekurrente Architektur

1:00 min / Lag-Sequenzen

Wir erweitern den Eingang des MLP um eine geordnete Historie. Zuerst werden hoch aufgelöste Messpunkte zu größeren Zeitintervallen zusammengefasst. Der Merkmalsvektor eines solchen Intervalls bildet einen Lag-Step. Mehrere aufeinanderfolgende Lag-Steps ergeben eine Lag-Sequenz.



Dissertation, Abbildung 5.2. Zeitliche Aggregation und Aufbau der Lag-Sequenz.

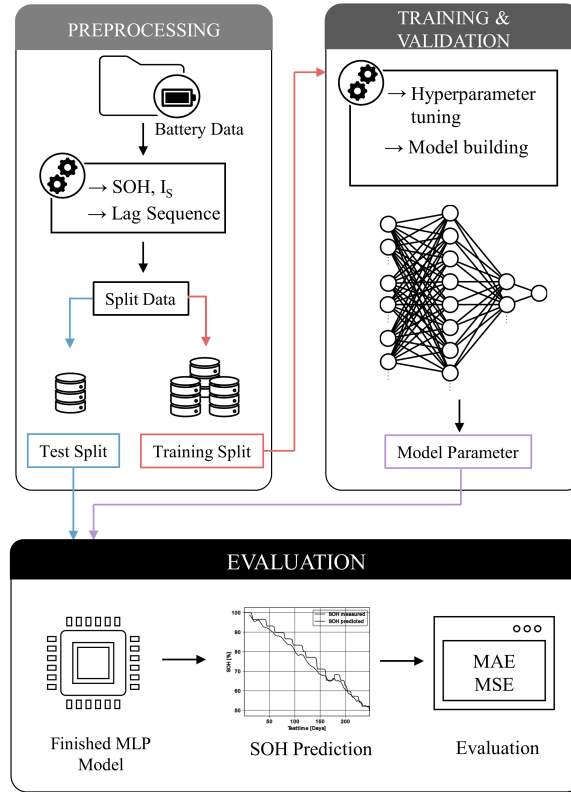
$$\mathbf{x}_k = [\phi_{k-L+1}^T \cdots \phi_k^T]^T, \quad \widehat{\text{SOH}}_k = f_\theta(\mathbf{x}_k). \quad (5)$$

Hier bezeichnet ϕ_k die Merkmale eines Zeitintervalls und L die Anzahl dieser Intervalle. Beispielsweise bilden 16 Schritte mit jeweils zehn Minuten eine Historie von 160 Minuten. Ein einzelner aggregierter Punkt ist also noch keine Sequenz. Das MLP verarbeitet die gesamte Historie als einen gemeinsamen Eingangsvektor. Die zeitliche Information entsteht hier durch die Datenaufbereitung, nicht durch einen internen Speicher des Netzes.

7 Training, Modellauswahl und unabhängige Evaluation

1:30 min / Methodischer Ablauf

Wie daraus ein belastbarer Modellvergleich entsteht, zeigt der Ablauf in Abbildung 5.3. Die Messdaten werden bereinigt, Merkmale gebildet und zeitlich geordnet. Die Skalierung bringt Eingänge mit unterschiedlichen Einheiten in einen geeigneten Wertebereich. Ihre Parameter werden aus den Trainingsdaten bestimmt und anschließend unverändert auf weitere Daten angewendet.



Dissertation, Abbildung 5.3. Datenaufbereitung, Training, Hyperparameteroptimierung und Evaluation.

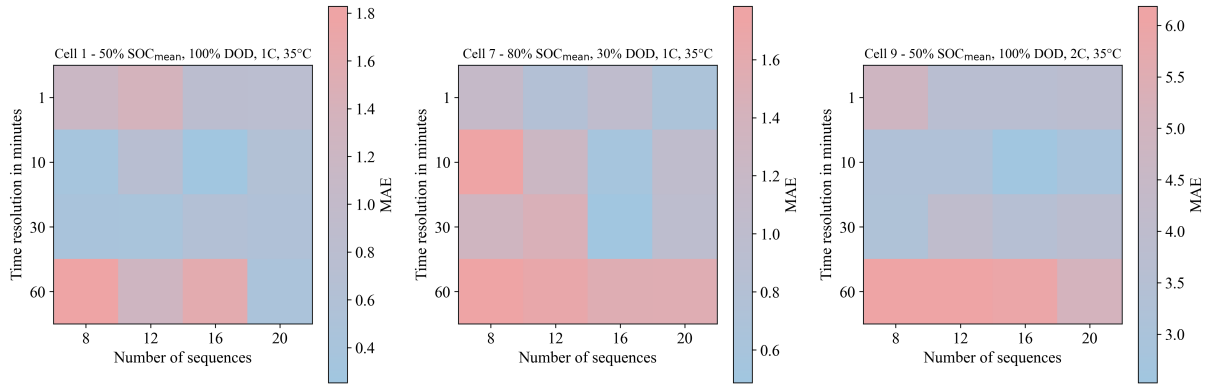
Training passt die Gewichte an. Validierung dient der Auswahl von Einstellungen wie Netzbreite, Sequenzlänge und Zeitauflösung. Die zur abschließenden Bewertung reservierten Testdaten dürfen diese Auswahl nicht steuern. Hyperparameteroptimierung untersucht daher mehrere Konfigurationen, statt nur ein Netz mit willkürlich gewählten Einstellungen zu trainieren.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (6)$$

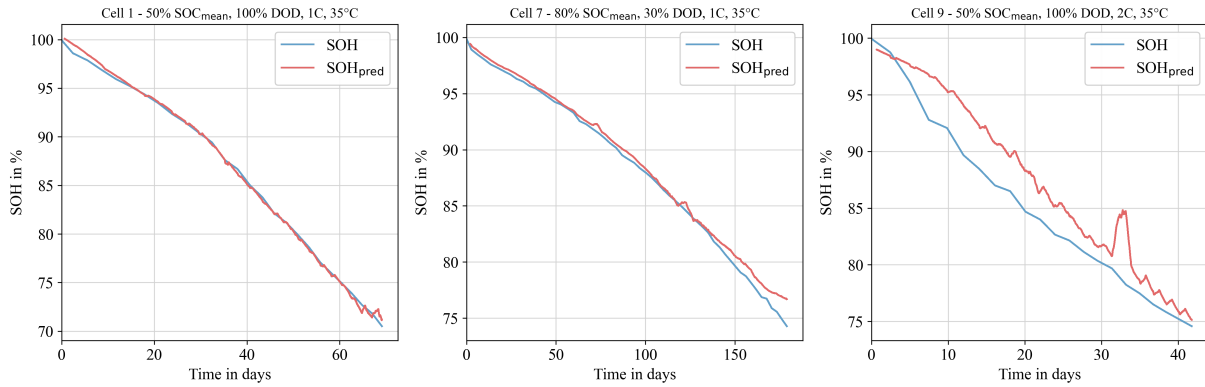
Der quadratische Fehler kann das Training steuern. Der MAE beschreibt die mittlere absolute Abweichung für die Evaluation. Bei einem auf null bis eins normierten SOH entsprechen 0,011 genau 1,10 Prozentpunkten. Die Trennung von Lernen, Auswahl und Prüfung bildet auch für die nachfolgenden neuronalen Modelle die methodische Grundlage.

8 Einfluss der Historie und erreichbare SOH-Schätzung

1:30 min / MLP-Ergebnisse



Dissertation, Abbildung 5.5. Fehlervergleich für unterschiedliche Kombinationen aus Lag-Länge und Zeitaufösung.



Dissertation, Abbildung 5.6. Vorhergesagter SOH und Referenzverläufe für ausgewählte Testfälle.

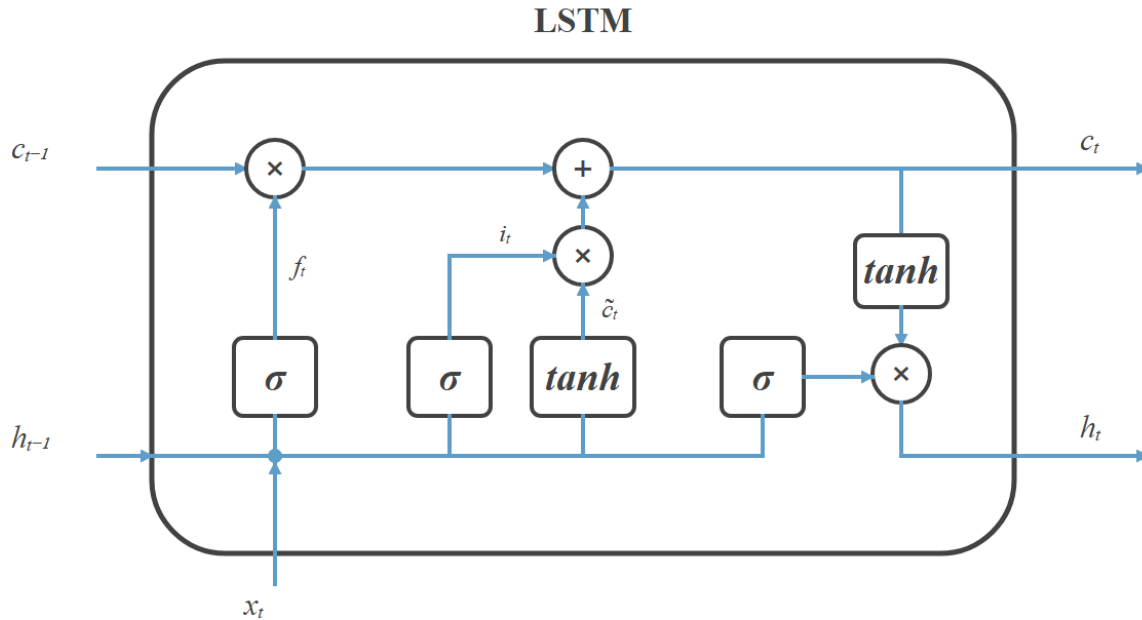
Die beiden Bilder beantworten zusammen die Ausgangsfrage. Erstens beeinflusst die Repräsentation das Ergebnis. Eine längere Historie ist nicht automatisch besser. In der untersuchten Suche ergab die Kombination aus 16 Schritten und zehn Minuten eine geeignete Konfiguration. Zweitens lässt sich der Alterungsverlauf bereits mit einer Feedforward-Architektur abbilden. Der mittlere Test-MAE liegt bei 1,10 Prozentpunkten, einzelne atypische Verläufe sind schwieriger.

Damit ist gezeigt, dass zeitliche Merkmale schon ohne rekurrente Zellen nützlich sind. Das ausgewählte MLP besitzt allerdings mehrere breite Schichten. Eine einfache *Architekturfamilie* bedeutet nicht automatisch wenig Parameter oder nachgewiesene MCU-Eignung. Der nächste Schritt untersucht deshalb eine andere Form der zeitlichen Verarbeitung und anschließend deren konkrete Implementierungskosten.

9 Von expliziten Lag-Sequenzen zu rekurrentem Kontext

1:45 min, beide Seiten / GRU und LSTM

Beim MLP mussten wir die Historie explizit zusammenstellen. Rekurrente Netze stellen eine wiederverwendbare Struktur zur Verfügung, in der zeitliche Information intern fortgeschrieben wird. Das vereinfacht die Modellierung einer fortlaufenden Verarbeitung. Es beseitigt jedoch weder Feature Engineering noch die Notwendigkeit, mit geeigneten Sequenzen zu trainieren.



Dissertation, Abbildung 3.3. LSTM mit Zellzustand, Hidden State und steuernden Gates.

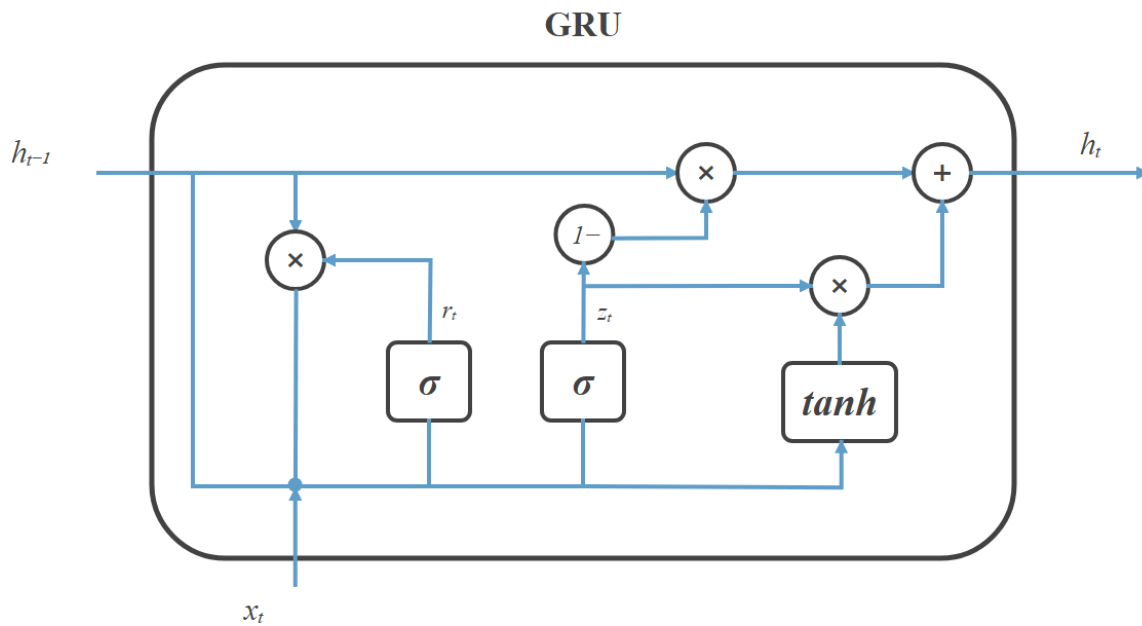
Beim LSTM entscheidet ein Forget Gate, wie viel des bisherigen Zellzustands erhalten bleibt. Ein Input Gate gewichtet neue Information. Ein Output Gate steuert, was davon als Hidden State weitergegeben wird:

$$\mathbf{c}_k = \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \mathbf{g}_k, \quad (7)$$

$$\mathbf{h}_k = \mathbf{o}_k \odot \tanh(\mathbf{c}_k). \quad (8)$$

Die Gate-Werte werden aus dem aktuellen Eingang und dem vorherigen Hidden State gelernt. $\odot$ bezeichnet elementweise Multiplikation. Der interne Zustand ersetzt damit nicht die Messdaten, sondern verdichtet deren bisherige Verarbeitung. Entscheidend ist anschließend, wann dieser Zustand weitergegeben oder zurückgesetzt wird.

GRU: zeitlicher Zustand mit weniger Gate-Struktur



Dissertation, Abbildung 3.4. GRU mit Reset Gate, Update Gate und einem fortgeschriebenen Hidden State.

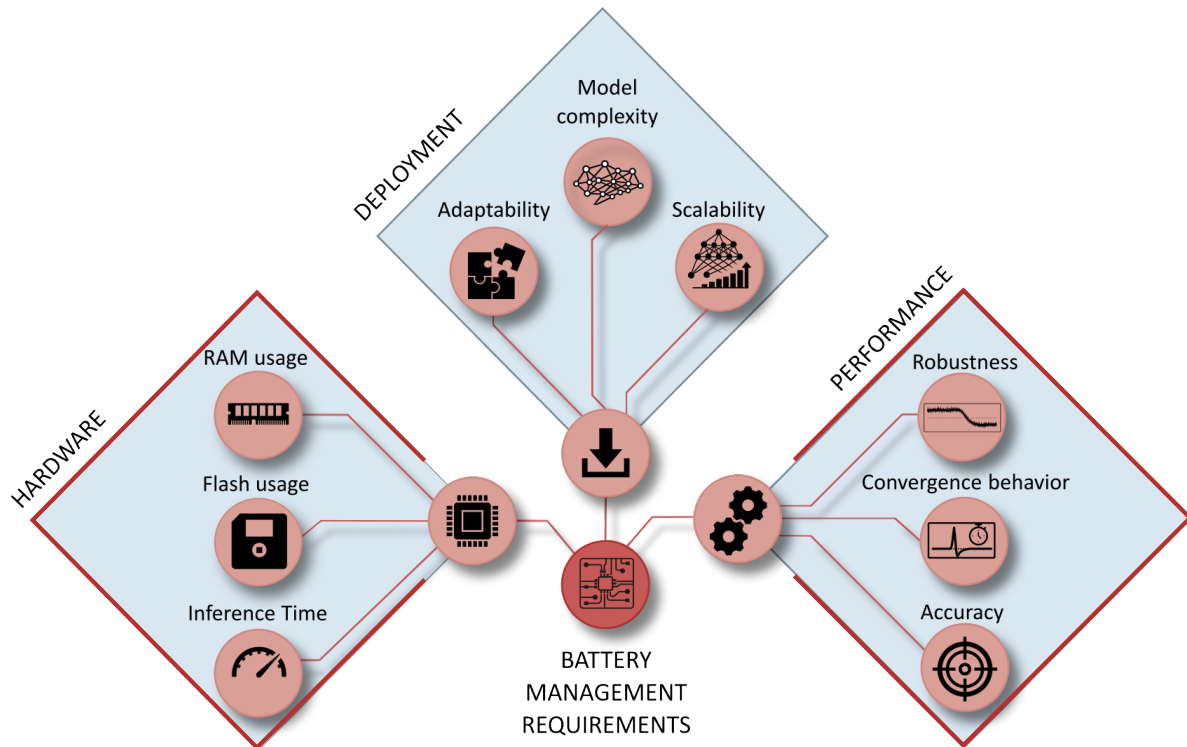
Eine GRU verwendet keinen zusätzlichen separaten Zellzustand. In der hier dargestellten Konvention mischt das Update Gate den bisherigen Zustand und einen neuen Kandidaten:

$$\mathbf{h}_k = (1 - \mathbf{z}_k) \odot \mathbf{h}_{k-1} + \mathbf{z}_k \odot \tilde{\mathbf{h}}_k. \quad (9)$$

Damit bieten GRU und LSTM zwei verwandte Wege, mehrere Messgrößen und ihren zeitlichen Zusammenhang zu verarbeiten. Der Wechsel vom MLP bedeutet also nicht, dass der erste Ansatz ungeeignet war. Er erweitert die Umsetzungsmöglichkeiten. Ob dieser gelernte Kontext im Betrieb tatsächlich hilfreich ist, zeigt erst der Vergleich unter realistischen Störungen. Und ob er kompakt bleibt, muss auf der Zielhardware geprüft werden.

Vorbereitung: Die vollständigen Gate-Gleichungen stehen in der Reserve. Im Vortrag an den beiden Bildern den Informationsfluss erklären, nicht jede Matrix vorlesen.

Robustheitsbenchmark: Verhalten und Ressourcen



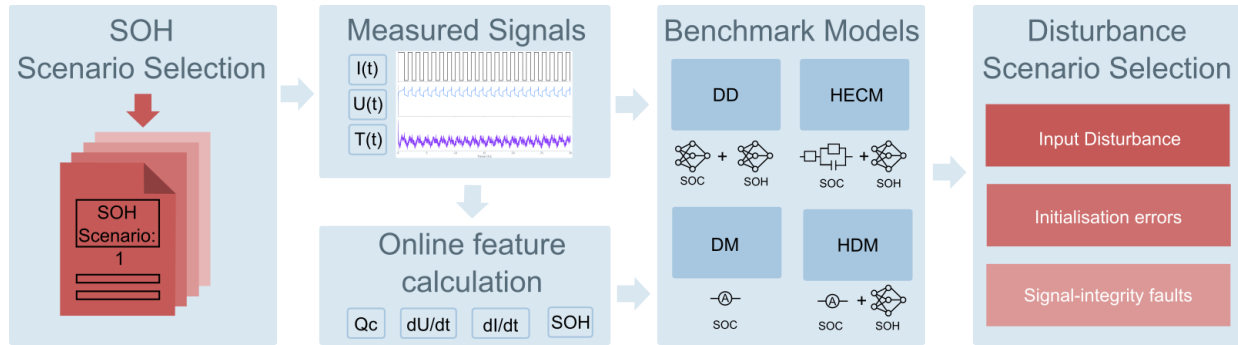
Dissertation, Abbildung 2.1. Schwerpunkt Performance und Hardware. Beide Bereiche werden nacheinander experimentell untersucht. Rot markiert den Hauptschwerpunkt.

Nach der zeitlichen Repräsentation richtet sich der Blick auf die Zuverlässigkeit im Betrieb. Wie verhalten sich unterschiedliche Zustandsbestimmungsmethoden bei veränderten Messbedingungen, und welche Ressourcen benötigen sie auf dem Mikrocontroller?

Zuerst vergleichen wir Genauigkeit, Robustheit und Erholung. Anschließend messen wir Flash, RAM und Laufzeit. Im Anforderungsbild werden daher Performance und Hardware gemeinsam hervorgehoben. Der Vergleich von Fensterbetrieb und kontinuierlicher Zustandsfortschreibung berührt zusätzlich Deployment, weil sich dabei die Ausführungsweise des neuronalen Modells ändert.

10 Vier Wege zur SOC-Bestimmung im selben Benchmark

1:30 min einschließlich Schwerpunktwechsel / Modellvergleich



Dissertation, Abbildung 6.1. Gemeinsamer Auswertungsablauf und vier SOC-Modellzweige.

Der Benchmark vergleicht unterschiedliche Rechenprinzipien. Direct Measurement, DM, integriert den Strom mit einer festen Kapazität. Hybrid Direct Measurement, HDM, verwendet zusätzlich einen geschätzten SOH zur Kapazitätsanpassung. Für positive Laderichtung und Zeit in Sekunden lässt sich der Kern schreiben als

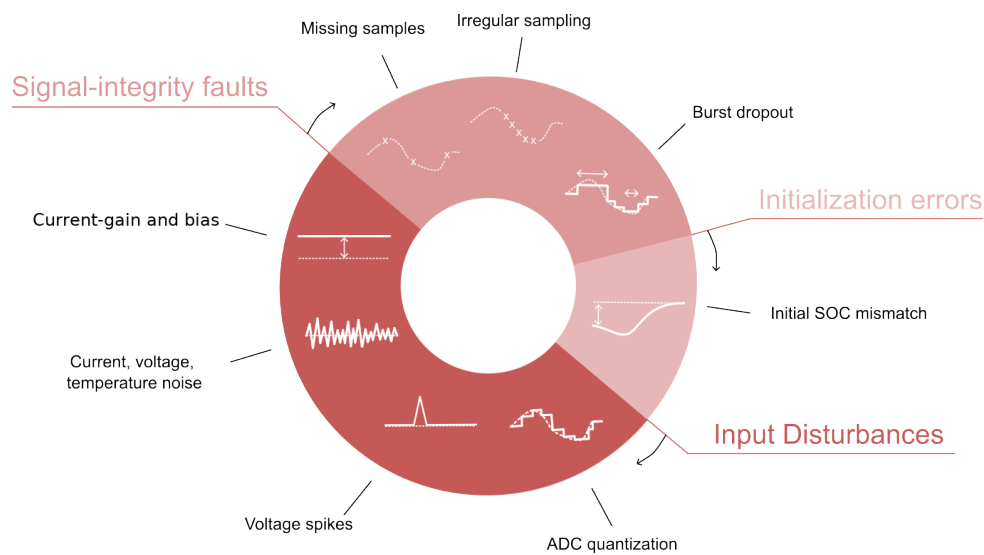
$$Q_{c,k} = Q_{c,k-1} + \frac{I_k \Delta t_k}{3600}, \quad \widehat{SOC}_k = \text{clip} \left(1 + \frac{Q_{c,k}}{C_{\text{eff},k}}, 0, 1 \right). \quad (10)$$

Q_c wird am Vollladeanker zurückgesetzt. Bei DM ist $C_{\text{eff}} = C_{\text{nominal}}$, bei HDM $C_{\text{eff}} = C_{\text{nominal}} \widehat{SOH}$.

Das hybride Ersatzschaltbildmodell HECM korrigiert seine modellbasierte Vorhersage zusätzlich anhand der Spannung. Der datengetriebene Zweig DD verwendet eine GRU mit mehreren Eingängen, darunter Spannung, Strom, Temperatur, SOH und zeitliche Ableitungen. HDM, HECM und DD erhalten denselben LSTM-basierten SOH-Schätzer. Unterschiede entstehen damit vor allem daraus, wie die SOC-Zweige die verfügbare Information verarbeiten.

11 Vom nominalen Test zu Betriebsstörungen

0:30 min / Benchmark-Design

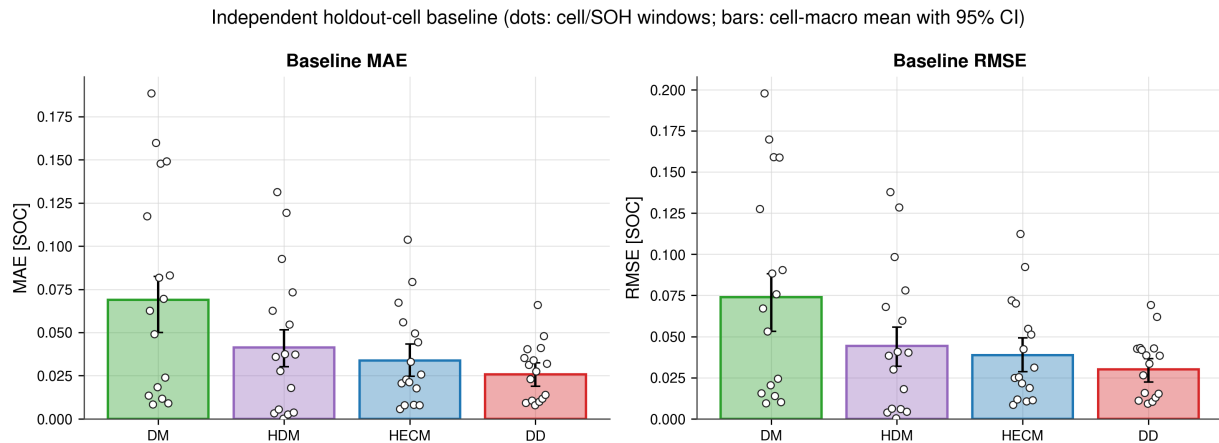


Dissertation, Abbildung 6.2. Störungsfamilien im kausalen Benchmark.

Die Testmatrix umfasst systematische Messfehler, zufälliges Rauschen, Initialisierungsfehler, fehlende Daten und lokale Ereignisse. Alle Modelle erhalten vergleichbar gestörte Eingänge. Abgeleitete Größen werden daraus neu berechnet. Die Auswertung folgt dem zeitlichen Verlauf und verwendet keine zukünftigen Messwerte. So untersuchen wir nicht nur Genauigkeit unter idealen Eingängen, sondern auch Stabilität und Erholung.

12 Nominale Genauigkeit als Ausgangspunkt

0:45 min / *Baseline*



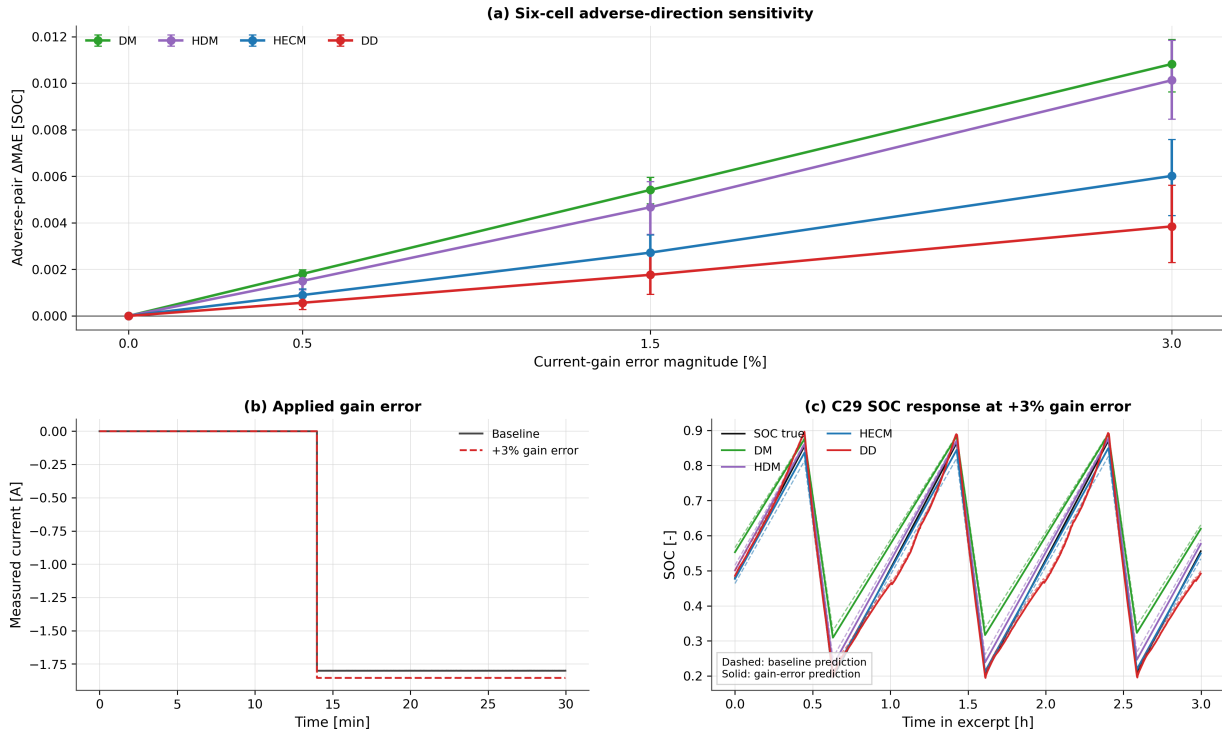
Dissertation, Abbildung 6.4. Baseline-Genauigkeit mit der Streuung zwischen den Testzellen.

Zunächst wird ohne zusätzliche Störungen ausgewertet. Grundlage sind 16 ausgewählte Zeitfenster aus sechs Testzellen. Die Zellen gehen gleichgewichtet in das Gesamtergebnis ein. Dieser Zellmittelwert des MAE beträgt etwa 6,90 Prozentpunkte für DM, 4,12 für HDM, 3,37 für HECM und 2,58 für DD. Die Einzelpunkte zeigen die Streuung, die Fehlerbalken das 95-Prozent-Konfidenzintervall des aggregierten Werts.

DD erzielt hier die niedrigste beobachtete mittlere Abweichung. Das ist eine Eigenschaft der untersuchten Modelle und Testdaten, noch keine allgemeine Rangfolge aller Vertreter dieser Familien. Der interessante nächste Schritt ist deshalb, ob diese Reihenfolge unter veränderten Messbedingungen erhalten bleibt und welche Fehlermechanismen dabei sichtbar werden.

13 Warum systematische Stromfehler kritisch sind

1:00 min / Current-Gain-Sensitivität



Dissertation, Abbildung 6.5. Einfluss eines multiplikativen Strommessfehlers auf den SOC-Fehler.

Ein Stromsensor kann einen Verstärkungsfehler, einen additiven Offset oder Rauschen aufweisen. Diese drei Anteile lassen sich unterscheiden durch

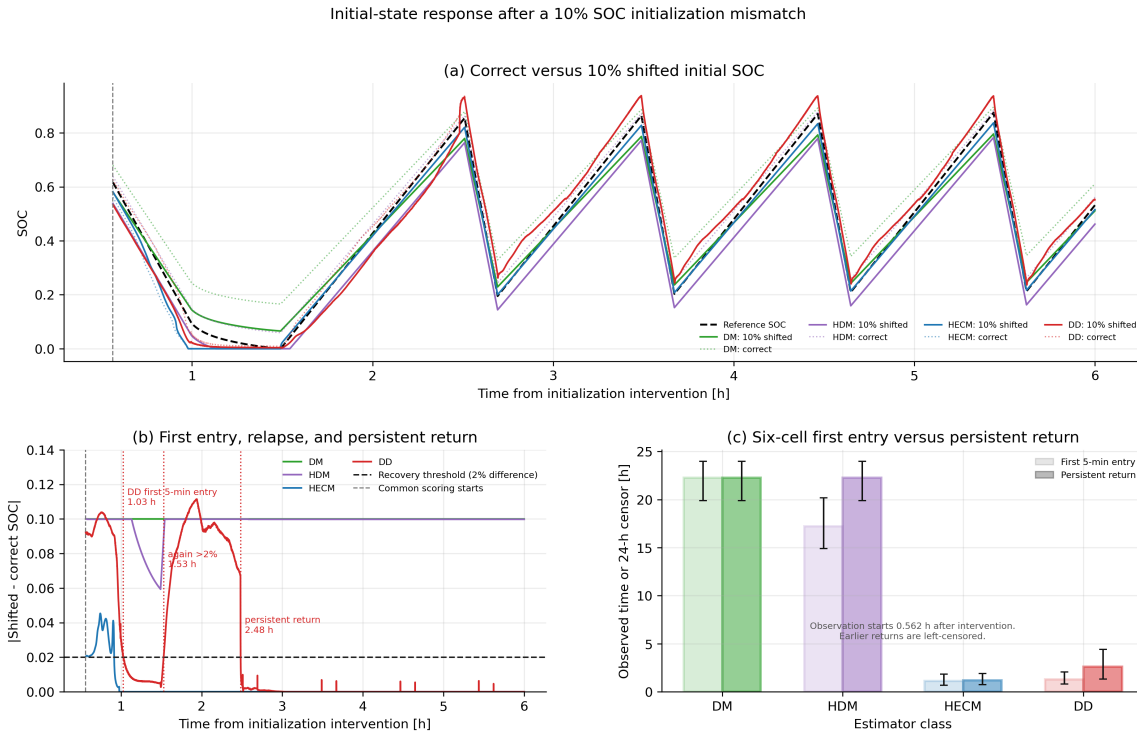
$$I_{\text{mess},k} = (1 + g)I_k + b + \varepsilon_k. \quad (11)$$

Abbildung 6.5 untersucht g , also den Gain. Bei drei Prozent steigt der MAE um etwa 1,08 Prozentpunkte für DM, 1,01 für HDM, 0,60 für HECM und 0,38 für DD. Ein systematischer Fehler im Strom geht bei der Ladungsbilanz fortlaufend in die aufsummierte Ladung ein.

HECM besitzt zusätzlich Spannungsrückkopplung, bleibt aber vom Strom abhängig. DD verarbeitet weitere Messgrößen und deren zeitlichen Kontext. Das bietet zusätzliche Information, ersetzt jedoch keinen fehlerfreien Sensor. Der additive Offset b wird separat untersucht und erweist sich in der Gesamtübersicht als besonders kritisch.

14 Erholung durch zusätzliche Zustandsinformation

1:00 min / Initialisierungsfehler



Dissertation, Abbildung 6.9. Erholung nach einer anfänglichen Zustandsabweichung.

Eine falsche Initialisierung verschiebt den Startpunkt der Schätzung. Reines Stromzählen kann diese Verschiebung nicht aus sich heraus erkennen. HECM verwendet dagegen die Differenz zwischen gemessener und vorhergesagter Spannung zur Korrektur:

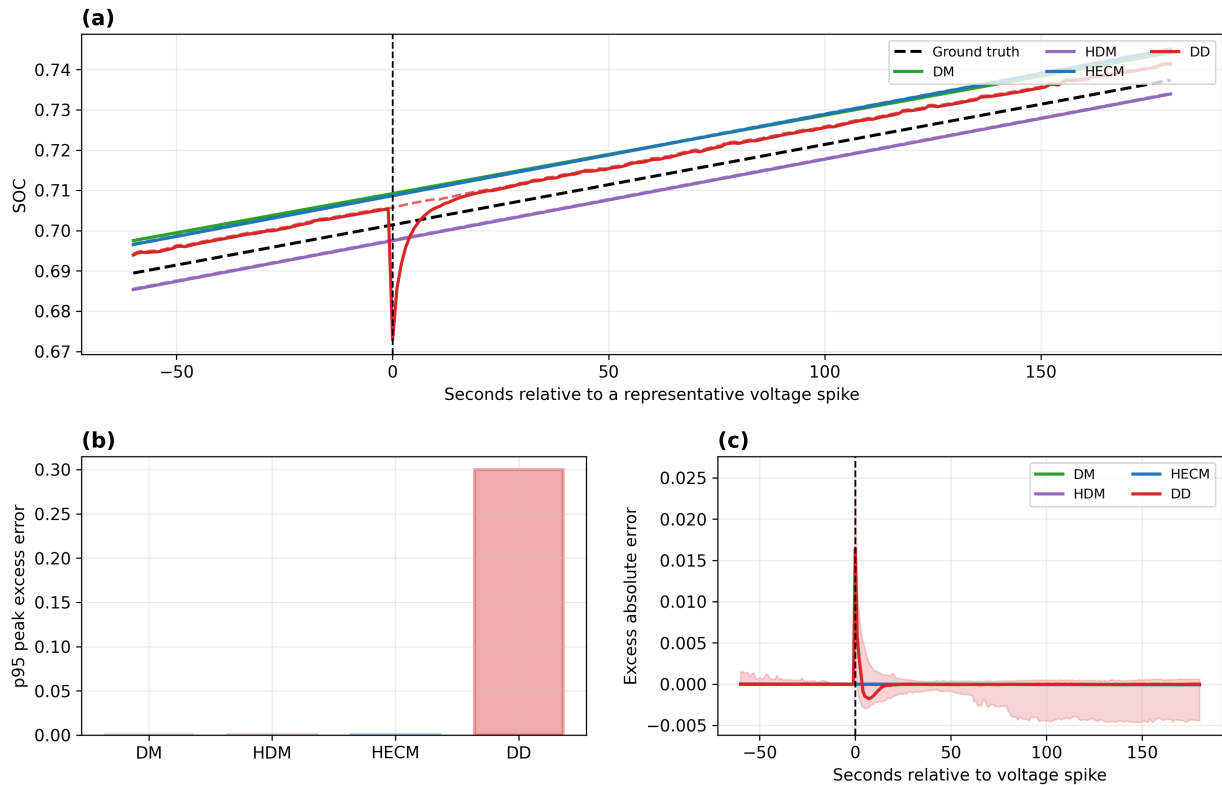
$$\mathbf{x}_k^+ = \mathbf{x}_k^- + \mathbf{K}_k \left(U_{\text{mess},k} - \hat{U}_k^- \right). \quad (12)$$

Die Zustandsgrößen umfassen SOC und die beiden RC-Spannungen. Die Gewichtung $\mathbf{K}_k$ legt fest, wie stark die Spannungsabweichung die Modellvorhersage korrigiert. Diese zusätzliche Rückkopplung erklärt, warum HECM trotz einer falschen Anfangslage wieder zum Referenzverlauf finden kann.

Beim untersuchten Fehler von minus zehn Prozentpunkten erreicht HECM das vorgegebene Erholungsband relativ zum ungestörten Vergleich dauerhaft nach im Mittel etwa 1,20 Stunden, DD nach 2,60 Stunden. DD kann den Zustand aus seiner mehrkanaligen Historie neu erschließen. Die Stromzähler benötigen dagegen einen geeigneten Anker. HECM zeigt hier also eine besondere Stärke, obwohl DD den kleineren nominalen MAE besitzt.

15 Lokale Ereignisse trotz kleinem Gesamtfehler

0:45 min / Spannungsspitze



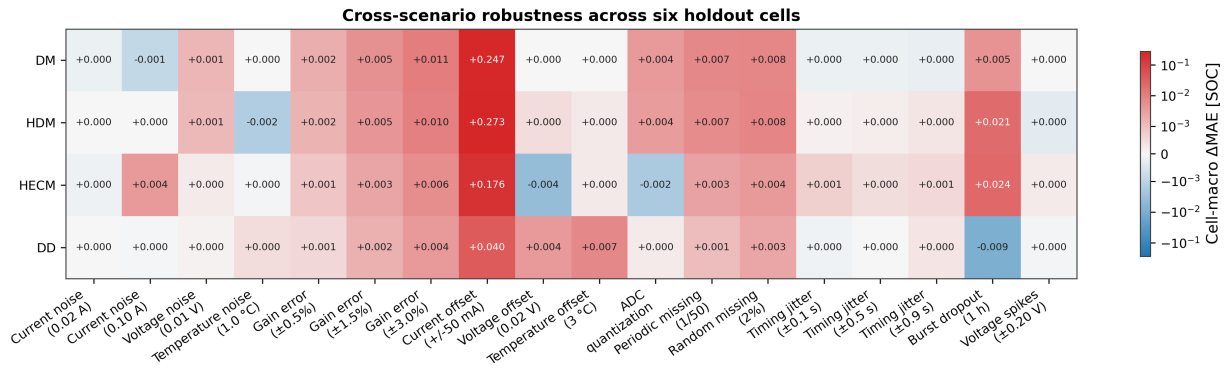
Dissertation, Abbildung 6.13. Spannungsspitze mit zeitaufgelöster Reaktion und ereignisbezogener Auswertung.

Eine kurze Spannungsspitze liefert ein anderes Bild. DD kann unmittelbar stark reagieren, obwohl der zusätzliche MAE über die gesamte Sequenz klein bleibt. Das ist mit der direkten Verarbeitung von Spannung und abgeleiteten dynamischen Merkmalen vereinbar. Der gelernte Kontext schützt also nicht automatisch vor jeder lokalen Eingabestörung.

DM und HDM verwenden die Spannung nicht als laufenden Korrekturinput. Ihre geringe Reaktion auf diesen Spannungsimpuls ist deshalb nicht einfach ein Mittelungseffekt der Stromintegration. Bei HECM wird die Spannungsinformation gewichtet in die Zustandskorrektur eingebracht. Das Beispiel zeigt, warum neben dem Gesamtfehler auch die unmittelbare Ereignisreaktion betrachtet werden muss.

16 Welche Störungen bestimmen das Gesamtbild?

0:30 min / Szenarioübersicht

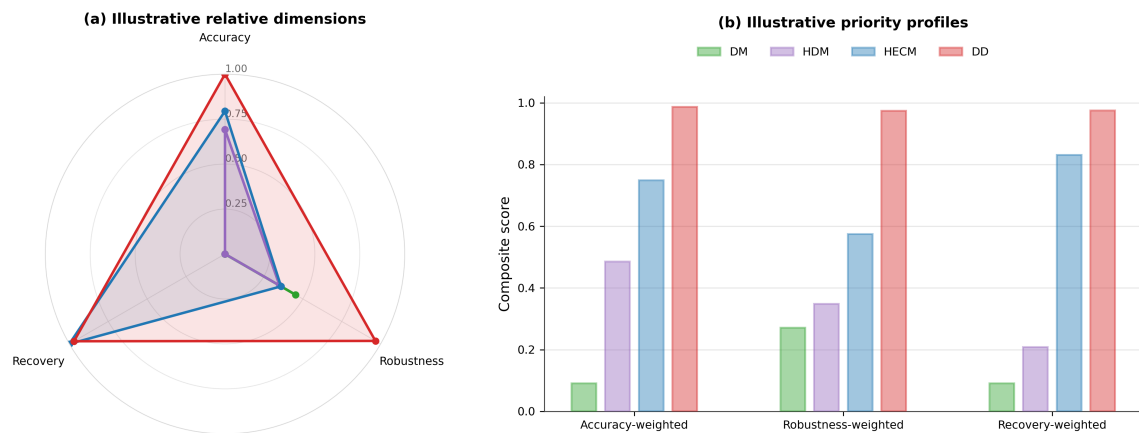


Dissertation, Abbildung 6.15. Fehlerzunahme über die untersuchten Störungsfamilien.

Die Übersicht ordnet die Einzelbeispiele ein. Vor allem systematische Stromfehler verursachen starke globale Abweichungen. Der additive Stromoffset fällt besonders auf. Kurze lokale Ereignisse können dagegen im globalen Mittel klein erscheinen und trotzdem eine ausgeprägte kurzzeitige Reaktion auslösen. Die beiden Betrachtungen beantworten unterschiedliche praktische Fragen und ergänzen sich.

17 Stärken der Modellfamilien statt einer einzigen Rangzahl

0:30 min | *Benchmark-Synthese*

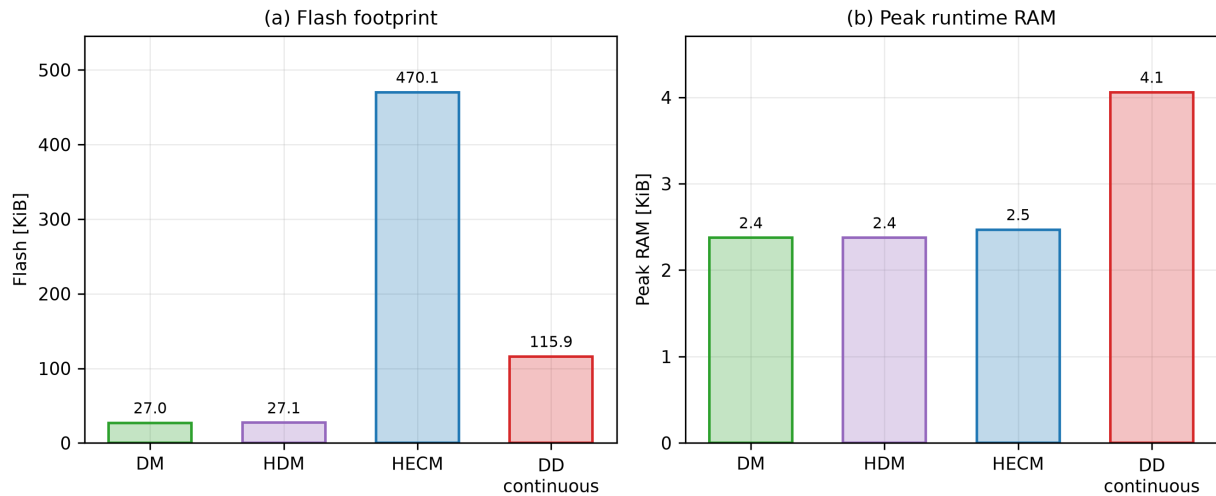


Dissertation, Abbildung 6.16. Synthese von Accuracy, Robustness und Recovery.

In der zusammengeführten Bewertung erzielt DD das stärkste Gesamtergebnis der untersuchten Vertreter. Die vorherigen Beispiele zeigen aber, warum die einzelnen Dimensionen erhalten bleiben müssen. HECM kann bei Erholung besonders gut abschneiden, während DD bei lokalen Eingangsspitzen empfindlich sein kann. Die normierten Scores im Bild sind relative Vergleichsgrößen, keine Genauigkeitsprozente. Für das BMS folgt daraus, ein ausgewähltes Modell gezielt auf seine relevanten Schwachstellen zu prüfen und darauf vorzubereiten.

18 Die Methoden auf dem Mikrocontroller

0:45 min / *Embedded-Performance*



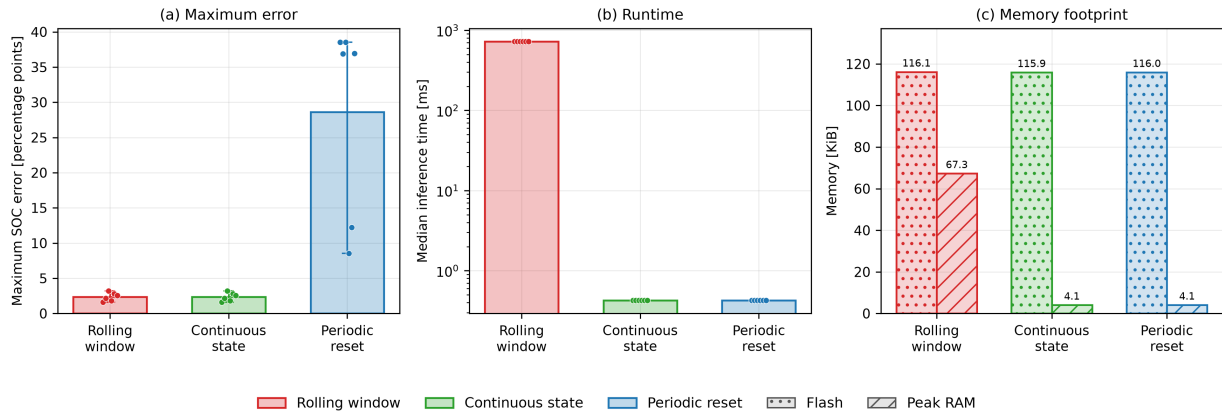
Dissertation, Abbildung 6.19. Flash und zur Laufzeit bestimmter RAM-Bedarf der isolierten SOC-Implementierungen.

Nun wechseln wir im Anforderungsrahmen von der Schätzgüte zur Ausführung. Alle vier SOC-Implementierungen wurden auf dem STM32H753 ausgeführt und mit ihren Software-Referenzen verglichen. Die Abweichungen sind sehr klein. Zugleich unterscheiden sich ihre Kosten deutlich.

DM und HDM benötigen ungefähr 27 KiB Flash, HECM rund 470 KiB und DD im kontinuierlichen Betrieb etwa 116 KiB. Der zugehörige Peak-RAM liegt zwischen 2,4 und 4,1 KiB. Alle passen auf den gewählten Controller. Gemessen werden hier die SOC-Kerne. Der benötigte SOH wird zugeführt, nicht mitsamt einem zusätzlichen SOH-Netz in diesen Zahlen versteckt.

19 Fensterbetrieb und kontinuierliche Zustandsfortschreibung

1:00 min / Ausführungsmodus



Dissertation, Abbildung 6.20. DD-Vergleich von Rolling Window, Continuous State und Periodic Reset.

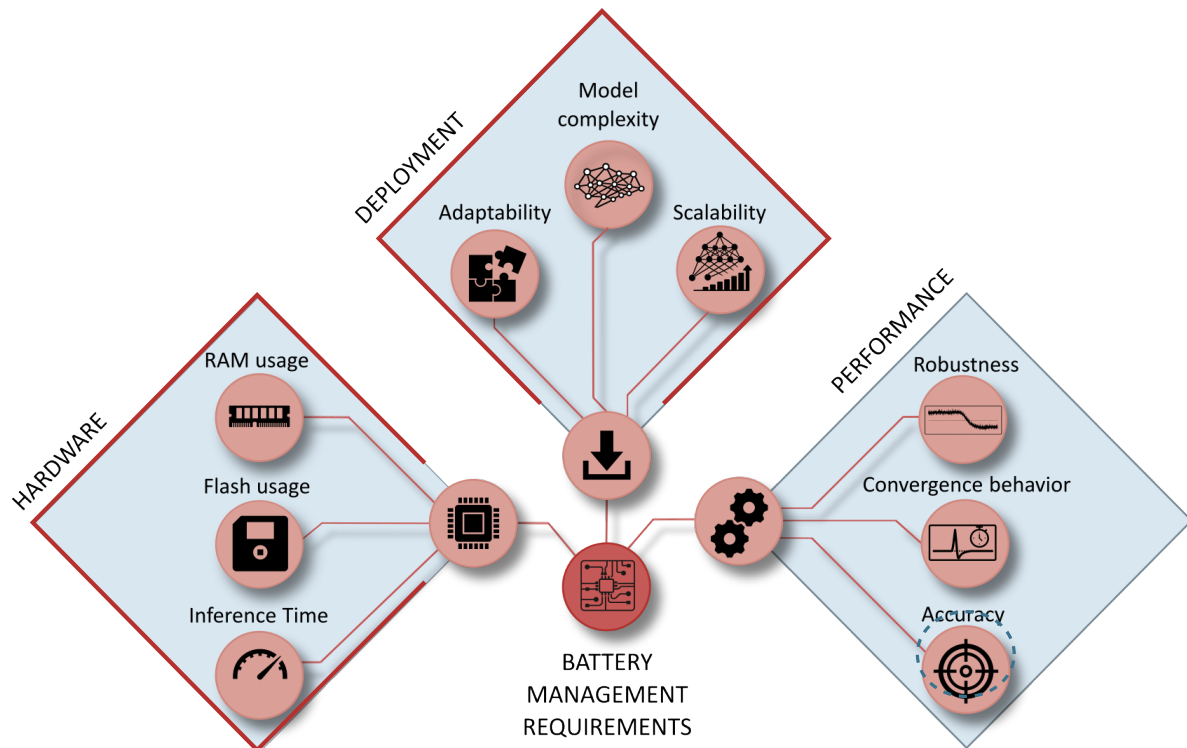
Im Rolling-Window-Betrieb wird für jede neue Schätzung erneut eine vollständige Eingangshistorie verarbeitet. Hier umfasst sie ungefähr 34 Minuten. Bei kontinuierlicher Ausführung wird dagegen nur der neue Schritt berechnet und der bisherige rekurrente Zustand weitergegeben:

$$\mathbf{h}_k = F_\theta(\mathbf{x}_k, \mathbf{h}_{k-1}), \quad \hat{y}_k = G_\theta(\mathbf{h}_k). \quad (13)$$

Das reduziert die gemessene DD-Laufzeit von etwa 724 Millisekunden auf 0,425 Millisekunden pro Ausgabe. Der Peak-RAM sinkt von rund 67,3 auf 4,1 KiB. Periodische Resets benötigen ebenfalls wenig Ressourcen, erzeugen jedoch wiederkehrende Anlauftransienten.

Kontinuierlicher Betrieb ist damit ein sehr wirksamer Hebel. Er ist aber nicht mathematisch identisch mit dem wiederholten Fensterlauf, weil die Zustandsvorgeschichte anders ist. Die gute nominale Übereinstimmung muss daher separat geprüft werden. Die Robustheitsaussagen des ursprünglichen Fensterbenchmarks gelten nicht ungeprüft für den anderen Ausführungsmodus.

Embedded-Optimierung: Modell und Implementierung



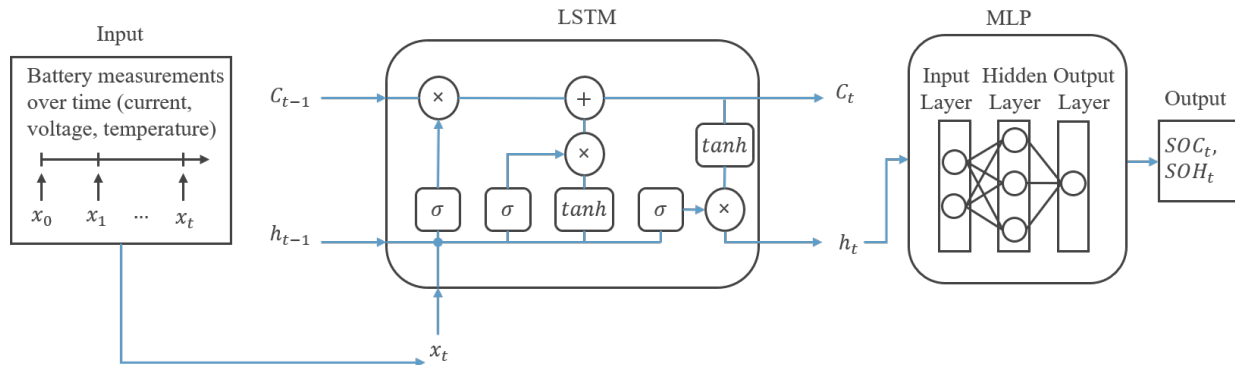
Dissertation, Abbildung 2.1. Schwerpunkt Deployment und Hardware. Die Genauigkeit bleibt als Kontrollgröße sichtbar. Rot markiert den Hauptschwerpunkt. Ein gestrichelter blauer Kreis markiert die ergänzend betrachtete Genauigkeit.

Der abschließende Teil verändert nun gezielt das Netz und seine Darstellung. Wie lässt sich eine funktionierende neuronale Zustandsbestimmung an begrenzte Hardwareressourcen anpassen, und welche Auswirkungen hat diese Optimierung auf die Schätzgüte?

Pruning reduziert die Struktur, Quantisierung die Darstellungsbreite ausgewählter Gewichte. Das sind Deployment-Entscheidungen. Ihre Wirkung messen wir anhand von Flash, RAM und Laufzeit. Accuracy bleibt mit im Bild, weil eine Ressourceneinsparung nur gemeinsam mit ihrer Auswirkung auf die Zustandsbestimmung beurteilt werden kann.

20 Welche Teile des Netzes lassen sich verkleinern?

1:00 min einschließlich Schwerpunktwechsel / LSTM-Architektur



Dissertation, Abbildung 7.1. LSTM mit nachgeschaltetem MLP als Ausgangspunkt der Kompression.

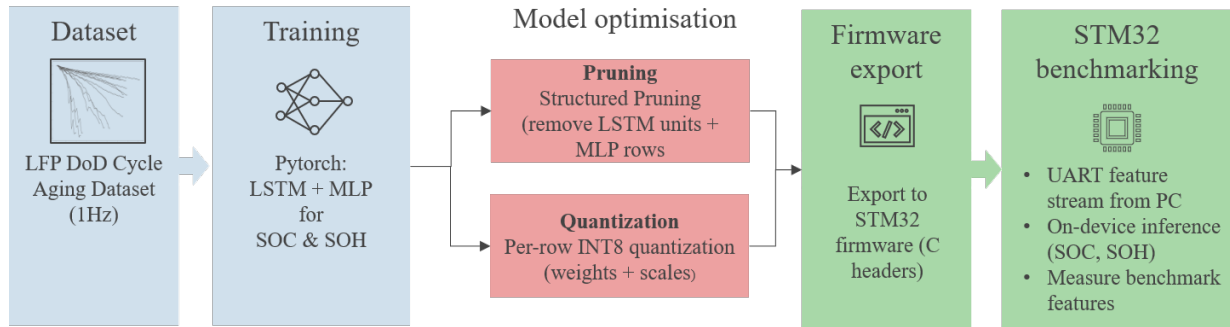
Der abschließende Teil untersucht nun die Architektur selbst. Hier werden eigenständige LSTM-Modelle für SOC und SOH verwendet, nicht unverändert die Modelle des vorherigen Benchmarks. Beide verarbeiten sechs Merkmale. Das SOC-Modell besitzt 64 rekurrente Einheiten, das SOH-Modell 128. Dahinter folgt ein MLP mit ebenfalls 64 beziehungsweise 128 Einheiten.

Architektur und Pruning	SOC	SOH
Eingangsdimension D	6	6
LSTM-Hidden-Size H , Base	64	128
LSTM-Hidden-Size, Pruned	45	90
MLP-Breite M	64	128
Ausgabe	SOC	SOH

Die Gewichte verbinden den aktuellen Eingang und den vorherigen Hidden State mit den Gates. Deshalb beeinflusst die rekurrente Breite mehrere Matrizen gleichzeitig. Hinzu kommen die Zustände im RAM und die Anbindung an das MLP. Diese Struktur erklärt, warum das Entfernen ganzer rekurrenter Einheiten sowohl Speicher als auch Rechenzeit verringern kann.

21 Von trainierten Parametern zu geprüfter Firmware

0:30 min / Optimierungsablauf

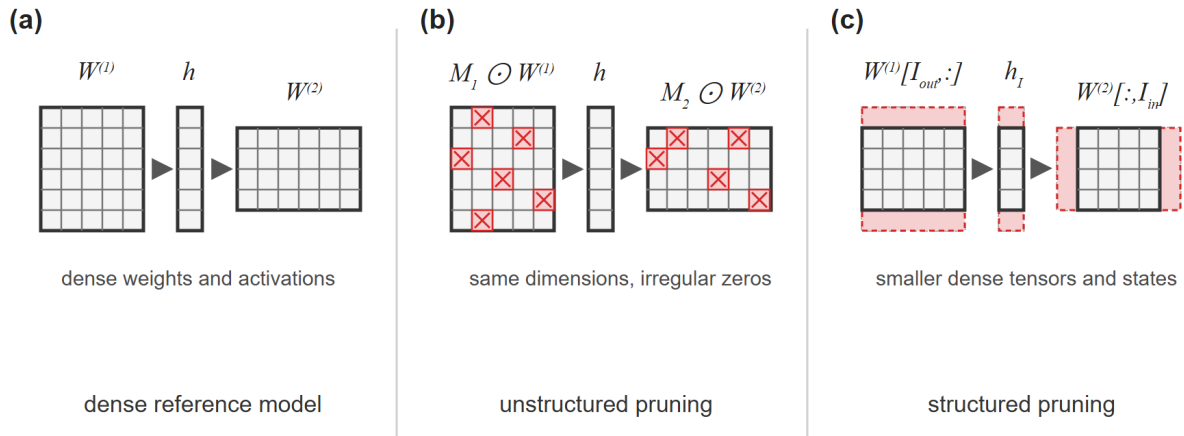


Dissertation, Abbildung 7.3. Training, alternative Kompressionspfade, C-Export und Hardware-Evaluation.

Aus dem trainierten FP32-Netz entstehen zwei getrennte Varianten. Beim strukturierten Pruning werden Einheiten entfernt und die verbleibenden Gewichte kurz nachtrainiert. Bei der Quantisierung werden ausgewählte Gewichtsmatrizen kompakter gespeichert. Anschließend werden die Parameter in C exportiert und auf derselben Hardware geprüft. So lassen sich Auswirkungen auf Genauigkeit, Speicher und Laufzeit gemeinsam betrachten, statt nur eine kleinere Modelldatei zu melden.

22 Strukturiertes Pruning und sein Rechenvorteil

1:30 min / Kompressionsgrundlagen



Dissertation, Abbildung 3.5. Unstrukturiertes Nullsetzen im Vergleich zum Entfernen ganzer Strukturen.

Einzelne Gewichte auf null zu setzen macht eine dichte Matrix noch nicht kleiner. Ohne passende Sparse-Kernel werden vielfach weiterhin dieselben Positionen verarbeitet. Strukturiertes Pruning entfernt dagegen ganze Einheiten samt ihren Verbindungen. Beim LSTM müssen die zugehörigen Gate-Zeilen, rekurrenten Spalten und MLP-Eingänge konsistent verkleinert werden.

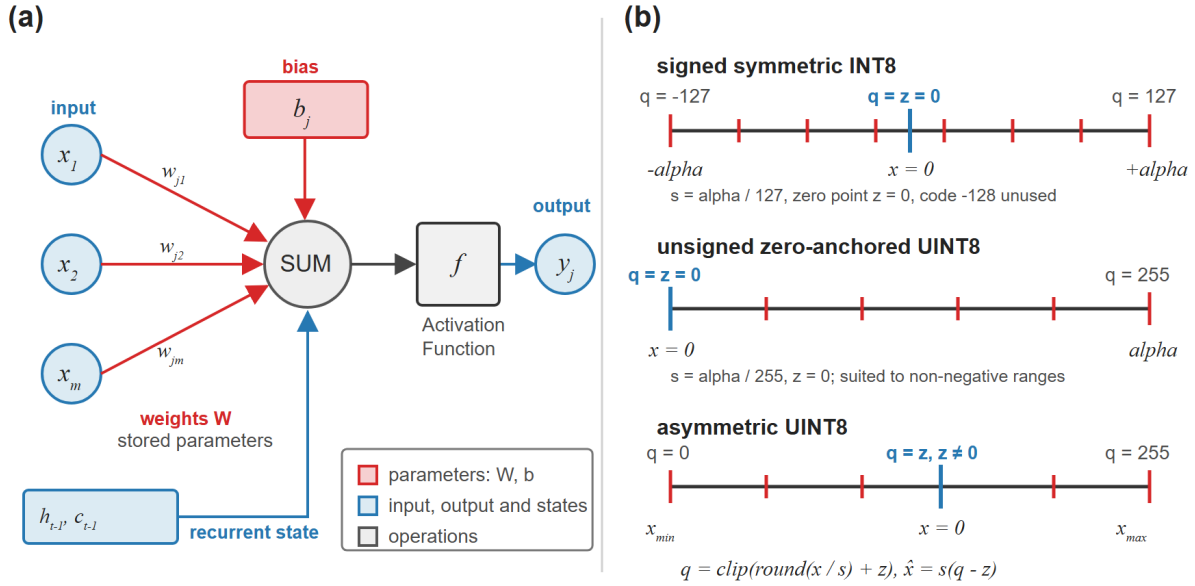
Die Auswahl basiert hier auf der Gewichtsnorm der rekurrenten Kanäle. Anschließend wird kurz nachtrainiert. Die Hidden Size sinkt um ungefähr 30 Prozent. Der Einfluss auf die Zahl der Multiply-Accumulate-Operationen, kurz MACs, lässt sich abschätzen:

$$N_{\text{MAC}} \approx 4H(D + H) + HM + M. \quad (14)$$

Vier affine LSTM-Transformationen erklären den ersten Term. Die folgenden Terme gehören zum MLP und zur Ausgabe. Da H auch quadratisch eingeht, kann der Rechenaufwand stärker sinken als die Hidden Size. Für SOC reduziert sich die Zahl von 22 080 auf 12 124 MACs pro Schritt, also um etwa 45 Prozent. Ob das auch schneller wird, prüfen wir anschließend mit realer Laufzeitmessung.

23 Quantisierung: weniger Bits, nicht automatisch weniger Laufzeit

1:15 min / Kompressionsgrundlagen



Dissertation, Abbildung 3.7. Quantisierungsumfang und Abbildung auf ein diskretes Wertegeraster.

Quantisierung verändert die Darstellung der Gewichte. Anstelle eines FP32-Werts werden ein ganzzahliger Wert und ein Skalierungsfaktor verwendet. Hier wird jede Zeile der rekurrenten Gewichtsmatrizen symmetrisch auf INT8 abgebildet:

$$s_r = \max\left(\frac{\max_j |w_{rj}|}{127}, \epsilon\right), \quad (15)$$

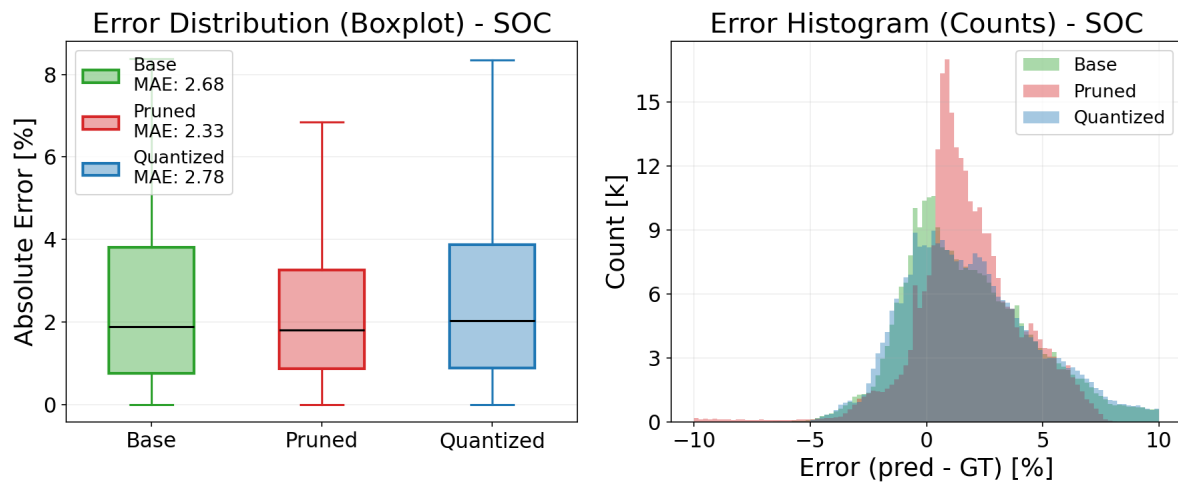
$$q_{rj} = \text{clip}\left(\text{round}\left(\frac{w_{rj}}{s_r}\right), -127, 127\right), \quad \hat{w}_{rj} = s_r q_{rj}. \quad (16)$$

Für diese gespeicherten Gewichte sinkt der Bedarf von vier auf ein Byte pro Wert, ergänzt um die Skalen. Das gesamte Netz wird dadurch aber nicht viermal kleiner. Zustände, Aktivierungen, Bias und MLP bleiben in diesem Ansatz FP32.

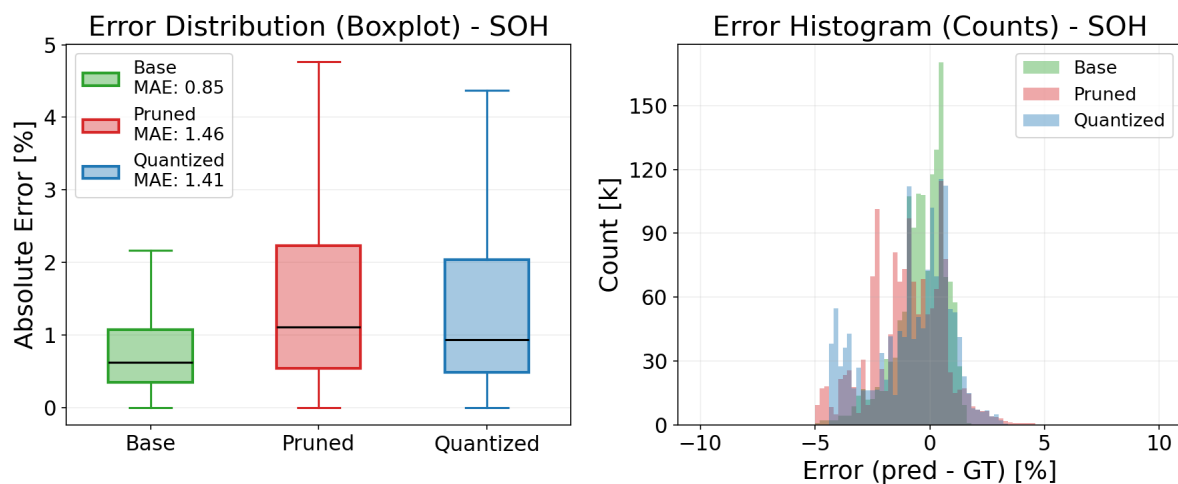
Auch die Rechnung bleibt gleitkommabasiert. Die kompakten Gewichte werden während der Berechnung rekonstruiert. Das spart Flash, kann aber zusätzliche Laufzeit kosten. Pruning reduziert also die Struktur, Quantisierung die Darstellungsbreite. Beides sind unterschiedliche Hebel. Ihre Kombination ist eine sinnvolle Weiterentwicklung, wurde hier aber nicht als dritte kombinierte Kompressionsvariante vermessen.

24 Wie viel Schätzgüte bleibt erhalten?

1:00 min / SOC- und SOH-Ergebnisse



Dissertation, Abbildung 7.8. SOC-Fehlerverteilungen der drei Deployment-Varianten.



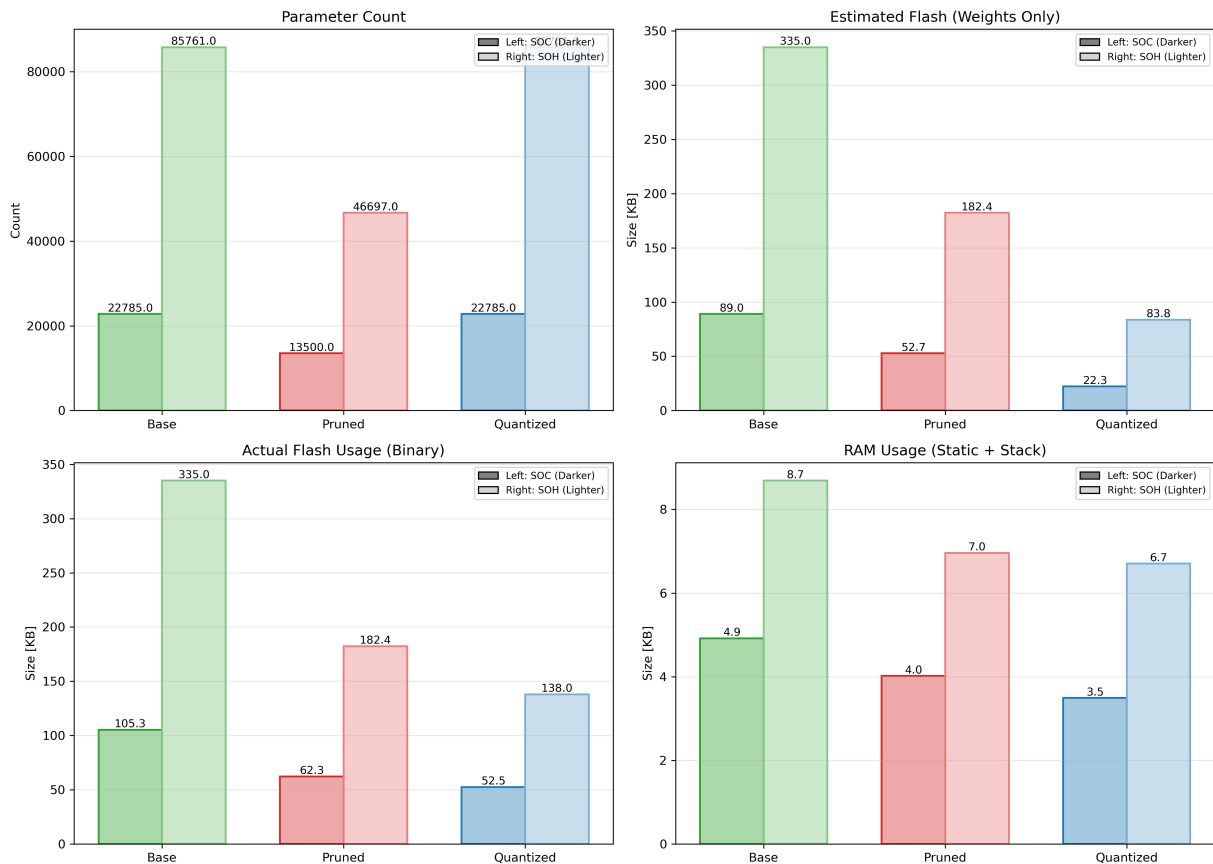
Dissertation, Abbildung 7.9. SOH-Fehlerverteilungen nach der jeweiligen kausalen Ausgabeverarbeitung.

Bei SOC bleiben die Verteilungen in einer ähnlichen Größenordnung. Die mittleren absoluten Fehler liegen bei ungefähr 2,7 Prozentpunkten für Base, 2,3 für Pruned und 2,8 für Quantized. Pruning verschlechtert das Ergebnis hier also nicht zwangsläufig. Das ist ein beobachtetes Ergebnis dieses Vergleichs, kein allgemeines Versprechen.

Bei SOH ist der Unterschied deutlicher. Base erreicht 0,85 Prozentpunkte, Pruned 1,46 und Quantized 1,41. Die komprimierten Modelle bleiben nutzbar, bezahlen ihre Ressourceneinsparung hier aber mit höherem Fehler. Die Werte beziehen sich auf die beschriebene kausale Nachverarbeitung. Gerade bei einem langsamen SOH-Signal müssen Glättung und Reaktionsverzögerung mitgedacht werden. Die Aussage lautet deshalb nicht, dass alle Varianten identisch sind, sondern dass sich unterschiedliche praktische Betriebspunkte erreichen lassen.

25 Parameter, Flash und RAM gemeinsam betrachten

0:45 min / Speicherbedarf



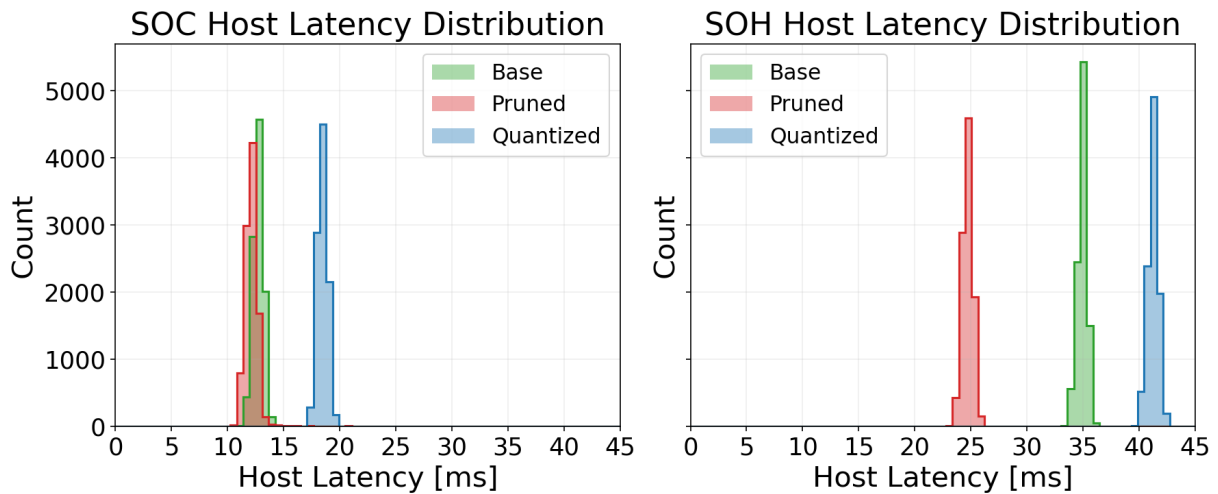
Dissertation, Abbildung 7.12. Parameterzahl, idealisierte Gewichtsspeicherung und Firmware-Ressourcen. Fachliche Prüfhinweise zur Originalgrafik stehen am Ende dieser Fassung.

Die Speicherauswertung trennt die reine Gewichtsdarstellung von der tatsächlichen Firmware. Pruning senkt den SOC-Flash von rund 105 auf 62 KB und den SOH-Flash von 335 auf 182 KB. Quantisierung erreicht mit ungefähr 52 und 138 KB die kleinsten Flash-Werte.

RAM umfasst dagegen auch Zustände, Puffer und den Stack. Deshalb folgt seine Reduktion nicht einfach derselben Quote wie die Gewichtsspeicherung. Die wesentliche Botschaft ist, dass ein Modell nicht nur anhand seiner Parameterzahl beurteilt werden kann. Entscheidend bleibt die gemessene oder aus der Firmware bestimmte Ressource auf der konkreten Zielpattform.

26 Weniger Rechenoperationen und gemessene Laufzeit

0:45 min / Inferenz und Gesamtlatenz



Dissertation, Abbildung 7.13. Verteilung der End-to-End-Latenz mit UART-Kommunikation und Hardware-Ausführung.

Abbildung 7.13 zeigt die Zeit vom Host-Aufruf bis zur Rückgabe des Ergebnisses. Darin steckt auch Kommunikation. Für die Optimierung des Netzes ist zusätzlich die direkt auf dem Controller gemessene Kernelzeit entscheidend:

Kernelzeit, Tabellen 7.1 und 7.2	Base	Pruned	Quantized
SOC [ms]	1,40	0,80	6,99
SOH [ms]	22,73	12,72	29,21

Pruning verkürzt die Netzberechnung bei beiden Aufgaben um ungefähr 43 bis 44 Prozent. Quantisierung spart am meisten Flash, benötigt in diesen gemischten INT8/FP32-Kernels aber länger. Welcher Betriebspunkt geeigneter ist, hängt davon ab, ob Speicher, Zeit oder Genauigkeit die engste Grenze setzt. Das lässt sich aus einem PC-MAE allein nicht entscheiden.

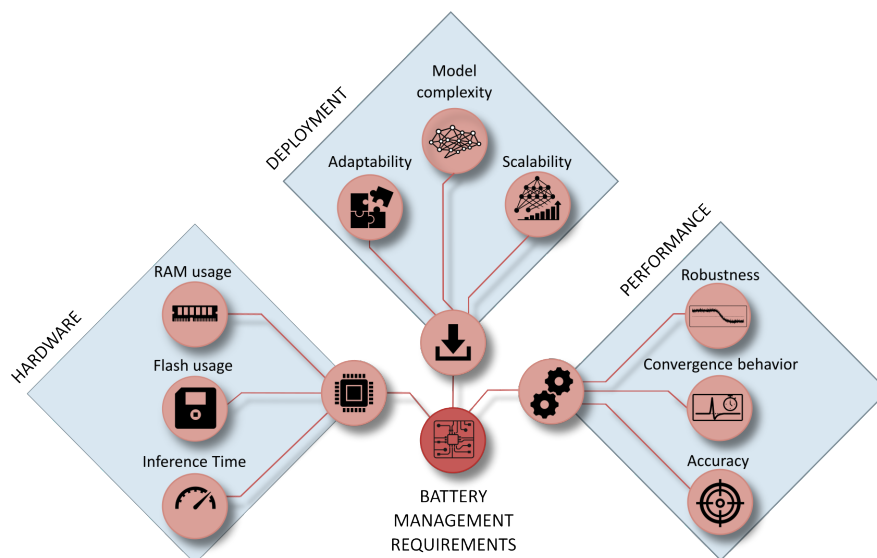
27 Gesamtergebnis und nächste Schritte

2:15 min / BMS-Anforderungen zusammenführen

Die Ergebnisse verbinden die drei Bereiche des Anforderungsbilds. Deployment beginnt bereits bei der Modellgestaltung. Das MLP-Beispiel zeigt, dass Merkmalsbildung und explizite Historie einen großen Beitrag leisten. Zeitabhängiges Batterieverhalten verlangt nicht automatisch die aufwendigste Netzstruktur. Rekurrente Modelle erweitern diesen Ansatz um einen intern fortgeschriebenen Zustand.

Performance beschreibt, wie verlässlich diese Modelle unter veränderten Messbedingungen arbeiten. Im Benchmark bietet der datengetriebene Ansatz ein starkes Gesamtergebnis. Das passt zur gemeinsamen Verarbeitung mehrerer Eingangsinformationen und ihres zeitlichen Kontexts. Zugleich bleiben spezifische Schwachstellen sichtbar. Systematische Stromfehler belasten die Ladungsbilanz. Spannungsrückkopplung hilft bei Erholung. Lokale Spannungsspitzen können gerade im neuronalen Modell ausgeprägte Reaktionen hervorrufen. Diese Befunde liefern Ansatzpunkte für geeignete Trainingsstörungen, Signalbehandlung und Zustandsinitialisierung.

Auf der Hardware werden die Folgen der Entwurfsentscheidungen messbar. Kontinuierliche Zustandsfortschreibung vermeidet die wiederholte Berechnung einer langen Historie. Strukturiertes Pruning reduziert die Dimension des Netzes und damit Speicher und Laufzeit. Quantisierung verdichtet die Gewichtsdarstellung, ihre Geschwindigkeitswirkung hängt jedoch von der Rechenimplementierung ab. Diese Maßnahmen müssen daher zusammen mit ihrer Auswirkung auf die Schätzung bewertet werden.

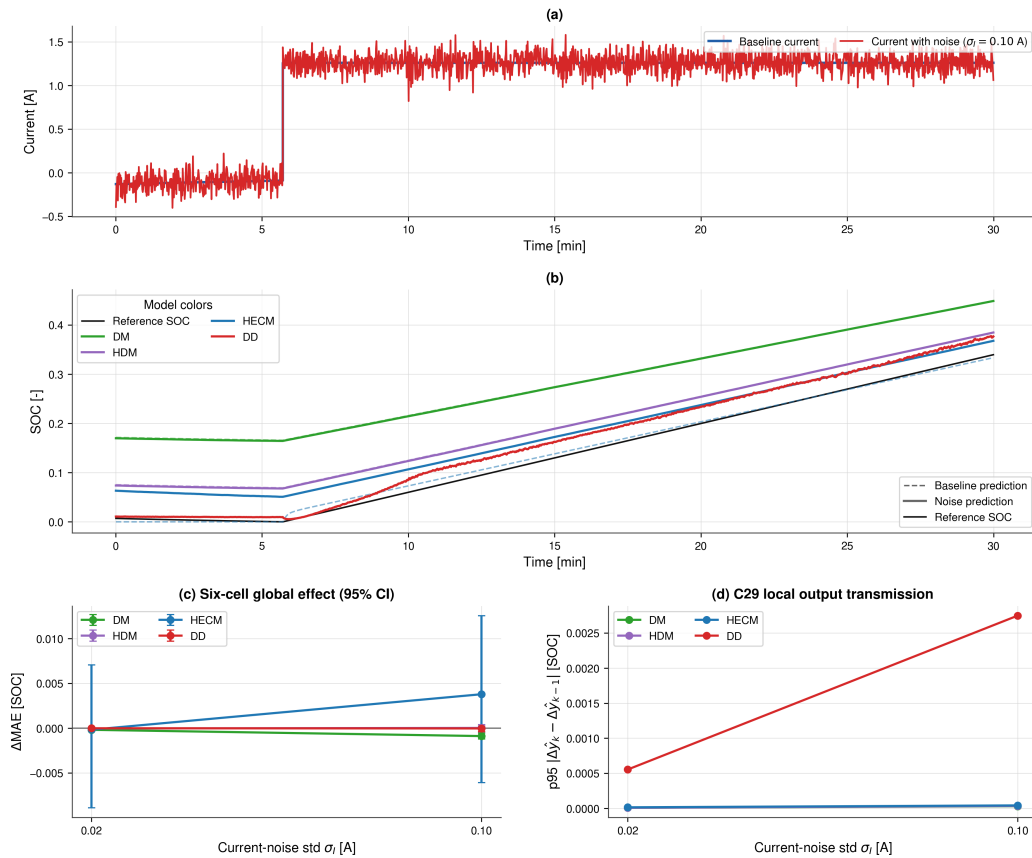


Dissertation, Abbildung 2.1. Rückkehr zum Anforderungsrahmen: Information, Modellverhalten und Implementierung treffen auf demselben BMS zusammen.

Der Gesamtbeitrag ist damit ein deploymentorientierter Weg vom Messsignal bis zur kompakten neuronalen Firmware. Die Netze sollen genug Komplexität besitzen, um das Batterieverhalten zu erfassen, aber keine unnötigen Ressourcen beanspruchen. Die Hardware-Ergebnisse zeigen, dass dies mit den untersuchten Modellen auf einem begrenzten Mikrocontroller erreichbar ist.

Als nächste Schritte bieten sich die gemeinsame Optimierung von Architektur und Kompression sowie Hardware-in-the-Loop-Tests mit gezielt gestörten Messsignalen auf der eigenen BMS-Plattform an. Hinzu kommen breitere Betriebsdaten und eine erneute Robustheitsprüfung der kontinuierlichen Ausführung. Damit wird die lokale Zustandsbestimmung nicht nur kompakter, sondern gezielter auf die Bedingungen vorbereitet, unter denen sie später verlässlich arbeiten soll.

Reserve: Warum Stromrauschen anders wirkt



Dissertation, Abbildung 6.7. Lokale Stromrauschreaktion und globaler Fehlervergleich. Reservebild zum Hauptbeispiel 6.13.

Stromrauschen und eine Spannungsspitze illustrieren unterschiedliche Mechanismen. Bei DM und HDM geht jeder verrauschte Stromwert in die Ladungsbilanz ein. Positive und negative Fehler können sich dabei teilweise ausgleichen. Es ist also nicht so, dass kleine neue Werte ignoriert werden. Ihr Beitrag wird integriert:

$$\Delta Q_{c,N} = \frac{\Delta t}{3600} \sum_{k=1}^N \varepsilon_k. \quad (17)$$

Bei unabhängigem, mittelwertfreiem Rauschen mit Varianz σ_I^2 gilt innerhalb eines Intervalls ohne Reset oder Begrenzung

$$\mathbb{E}[\Delta Q_{c,N}] = 0, \quad \text{Var}(\Delta Q_{c,N}) = N \sigma_I^2 \left(\frac{\Delta t}{3600} \right)^2. \quad (18)$$

Die Standardabweichung wächst dann mit $\sqrt{N}$. Ein konstanter Offset liefert dagegen einen gerichteten Beitrag $Nb\Delta t/3600$. Zufallsrauschen verschwindet somit nicht vollständig, erzeugt aber einen anderen Fehlerverlauf als ein anhaltender Offset.

DD verarbeitet Strom und Stromableitung direkt. Dadurch kann mehr kurzfristige Variation im Ausgang sichtbar sein, ohne dass der globale MAE stark steigt. HECM nutzt Strom auch in der Zustands- und Spannungsvorhersage. Ein Kalmanfilter ist daher nicht grundsätzlich gegen jede Form von Messrauschen unempfindlich.

Auswahl für 30 Minuten. Abbildung 6.13 bleibt im Hauptteil, weil sie den Unterschied zwischen globaler Genauigkeit und lokalem Ereignis besonders deutlich macht. Abbildung 6.7 ist sinnvoll als Reserve oder als Ersatz, wenn die Stromintegration im Vortrag stärker erklärt werden soll. Beide vollständig zu präsentieren würde zusätzliche Zeit benötigen.

Reserve: Gate-Gleichungen und Parameter

LSTM. Aktueller Eingang $\mathbf{x}_k$, vorheriger Hidden State $\mathbf{h}_{k-1}$ und Zellzustand $\mathbf{c}_{k-1}$ bestimmen den nächsten Schritt:

$$\mathbf{f}_k = \sigma(W_f \mathbf{x}_k + U_f \mathbf{h}_{k-1} + \mathbf{b}_f), \quad (19)$$

$$\mathbf{i}_k = \sigma(W_i \mathbf{x}_k + U_i \mathbf{h}_{k-1} + \mathbf{b}_i), \quad (20)$$

$$\mathbf{g}_k = \tanh(W_g \mathbf{x}_k + U_g \mathbf{h}_{k-1} + \mathbf{b}_g), \quad (21)$$

$$\mathbf{o}_k = \sigma(W_o \mathbf{x}_k + U_o \mathbf{h}_{k-1} + \mathbf{b}_o), \quad (22)$$

$$\mathbf{c}_k = \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \mathbf{g}_k, \quad (23)$$

$$\mathbf{h}_k = \mathbf{o}_k \odot \tanh(\mathbf{c}_k). \quad (24)$$

Die Sigmoidfunktion $\sigma(z) = 1/(1 + e^{-z})$ beschränkt die Gates auf null bis eins. Der Kandidat $\mathbf{g}_k$ ist neue Information, kein viertes Sigmoid-Gate.

GRU. Passend zur Darstellung in Abbildung 3.4 wird hier folgende Reset-before-Konvention verwendet:

$$\mathbf{z}_k = \sigma(W_z \mathbf{x}_k + U_z \mathbf{h}_{k-1} + \mathbf{b}_z), \quad (25)$$

$$\mathbf{r}_k = \sigma(W_r \mathbf{x}_k + U_r \mathbf{h}_{k-1} + \mathbf{b}_r), \quad (26)$$

$$\tilde{\mathbf{h}}_k = \tanh(W_h \mathbf{x}_k + U_h(\mathbf{r}_k \odot \mathbf{h}_{k-1}) + \mathbf{b}_h), \quad (27)$$

$$\mathbf{h}_k = (1 - \mathbf{z}_k) \odot \mathbf{h}_{k-1} + \mathbf{z}_k \odot \tilde{\mathbf{h}}_k. \quad (28)$$

Softwarebibliotheken können das Update Gate umgekehrt definieren oder den Reset nach der rekurrenten Matrixoperation anwenden. Beim C-Export ist deshalb die konkrete Operatorfolge des trainierten Modells maßgeblich. Das vereinfachte Zellbild ersetzt keine Implementierungsspezifikation.

Speicherwirkung. Bei Eingangsdimension D , Hidden Size H und einem zusammengefassten Bias pro Transformation sind die Parameterzahlen der rekurrenten Schicht

$$N_{\text{LSTM}} = 4H(D + H + 1), \quad N_{\text{GRU}} = 3H(D + H + 1). \quad (29)$$

Bei zwei getrennt gespeicherten Bias-Vektoren pro Transformation wird aus +1 jeweils +2. Hinzu kommen Ausgabe- und gegebenenfalls weitere Schichten. Die reinen FP32-Zustände beanspruchen $8H$ Byte beim LSTM und $4H$ Byte bei der GRU. Das ist kein vollständiger RAM-Wert, da Puffer, Stack und weitere Firmware hinzukommen.

Reserve: Repräsentation und Optimierung

Belastungshistorie als Merkmal. Bei positiver Laderichtung und Zeitintervallen in Sekunden sind kumulierte Lade- und Entlademengen in Ah

$$Q_{+,k} = Q_{+,k-1} + \frac{\max(I_k, 0) \Delta t_k}{3600}, \quad (30)$$

$$Q_{-,k} = Q_{-,k-1} + \frac{\max(-I_k, 0) \Delta t_k}{3600}. \quad (31)$$

Anders als die vorzeichenbehaftete Ladungsbilanz enthalten beide Größen den jeweiligen akkumulierten Durchsatz. Die hohe Korrelation mit Alterung motiviert ihren Einsatz, kann aber zugleich durch die gemeinsame Entwicklung mit der Versuchsdauer geprägt sein.

Skalierung. Für das Min-Max-Prinzip gilt

$$\tilde{x} = \frac{x - x_{\min, \text{train}}}{x_{\max, \text{train}} - x_{\min, \text{train}}}. \quad (32)$$

Die Embedded-LSTM-Modelle verwenden dagegen die robuste Skalierung mit Trainingsmedian m und Interquartilsabstand s , also $\tilde{x} = (x - m)/s$. Beide Verfahren lernen ihre Skalierungsparameter aus Trainingsdaten. Das Prinzip ist gemeinsam, das konkrete Verfahren muss je Modell erhalten bleiben.

Pruning-Kriterium. Die Bewertung eines rekurrenten Kanals h fasst die Normen seiner Input- und rekurrenten Gewichtszeilen über die vier LSTM-Blöcke zusammen:

$$s_h = \sum_{a \in \{i, f, o, g\}} \left(\|W_{ih}^{(a)}[h, :]\|_2 + \|W_{hh}^{(a)}[h, :]\|_2 \right). \quad (33)$$

Für die Menge $\mathcal{R}$ beibehaltener Kanäle werden neue dichte Matrizen gebildet:

$$\widetilde{W}_{ih}^{(a)} = W_{ih}^{(a)}[\mathcal{R}, :], \quad (34)$$

$$\widetilde{W}_{hh}^{(a)} = W_{hh}^{(a)}[\mathcal{R}, \mathcal{R}], \quad (35)$$

$$\widetilde{W}_{\text{MLP},1} = W_{\text{MLP},1}[:, \mathcal{R}]. \quad (36)$$

So wird aus dem allgemeinen Pruning-Bild eine konkrete, konsistente Verkleinerung der rekurrenten Implementierung.

Gewichtsspeicherung. Für N_W Gewichte mit b Bits ist der reine Speicher $B_W = N_W b / 8$ Byte. Im gemischten Export kommen FP32-Skalen, FP32-Teile und Programmcode hinzu. Darum ist die ideale Vierfachreduktion einzelner Matrizen nicht die Reduktion der gesamten Firmware.

Bildauswahl und Weg in die Folien

Die Originalabbildungen bleiben die Belege. Für die spätere PowerPoint werden bei dichten Mehrfachdiagrammen gezielt Teilpanels vergrößert. Es werden keine neuen Ergebniswerte oder vereinfachten Ersatzkurven erfunden. Neue Elemente dienen lediglich der Orientierung, zum Beispiel der Hervorhebung eines Pfads im BMS-Anforderungsbild.

Station	Originalabbildungen	Aufgabe im Vortrag
1, 27	2.1	Gemeinsamer Rahmen und Rückbindung.
Vor 4	2.1, Fokusansicht	Deployment und Modellkomplexität, Accuracy ergänzend.
2, 3	4.1, 4.4, 4.6	Datenvielfalt und reale BMS-Zielumgebung.
4 bis 7	3.2, 5.1, 5.2, 5.3	Netzoperation, Merkmale, Historie und saubere Evaluation.
8	5.5 und 5.6 gemeinsam	Repräsentationswahl und erreichbare SOH-Schätzung.
9	3.3 und 3.4	Interne Zeitverarbeitung statt explizitem MLP-Historienvektor.
Vor 10	2.1, Fokusansicht	Performance und Hardware.
10, 11	6.1 und 6.2	Modellfamilien und Testfamilien.
12 bis 15	6.4, 6.5, 6.9, 6.13	Baseline, Stromfehler, Erholung und lokales Ereignis.
16, 17	6.15 und 6.16	Einordnung aller Szenarien und Synthese.
18, 19	6.19 und zusätzlich 6.20	Ressourcen und kontinuierliche Ausführung.
Vor 20	2.1, Fokusansicht	Deployment und Hardware, Accuracy als Kontrollgröße.
20, 21	7.1, Daten aus 7.2, 7.3	Konkrete Architektur und Optimierungspfade.
22, 23	3.5 und 3.7	Technische Grundlagen direkt vor ihren Ergebnissen.
24	7.8 und zusätzlich 7.9	SOC und SOH getrennt beurteilen.
25, 26	7.12, 7.13, Tabellen 7.1/7.2	Speicher, Gesamtlatenz und reine Inferenz.
Reserve	6.7	Stromrauschen als ergänzender Mechanismus.

Lesbarkeit. Bei 4.4 nur den relevanten Zellverlauf und die Profizuordnung fokussieren. Bei 5.5 und 5.6 dieselbe Seite beibehalten, aber die Heatmap und einen Ergebnisverlauf nacheinander hervorheben. Bei 6.5 die Fehlerzunahme und den SOC-Verlauf auswählen. Bei 6.13 den Zeitverlauf und die ereignisbezogene Reaktion gegenüberstellen. Bei 7.8 und 7.9 jeweils zuerst den Boxplot zeigen. Das ganze Original dient als Kontext, ein vergrößerter Ausschnitt macht den Befund lesbar.

Navigation. Für die Folien den seitlichen Fortschrittsbalken aus v3 aufnehmen. Themen heißen Anforderungen, Daten, zeitliche Repräsentation, Robustheit und Embedded-Optimierung. Keine Aufteilung in nummerierte Studien. Die drei Fokusansichten stehen vor MLP, Benchmark und Optimierung auf den Seiten 5, 13 und 24. Am Hardware-Übergang innerhalb des Benchmarks wird der linke Bereich zusätzlich betont. Der Rest des Rahmens bleibt als Kontext sichtbar.

Typografie. Spätere Folien mit Arial, Titeln in 24 pt und Text in 18 pt. Dichte Originaldiagramme nicht beliebig verkleinern. Für sichtbare Diagrammtexte gelten dieselben Lesbarkeitsanforderungen. Kein dauerhafter roter Fazitsatz am Folienende.

Fachliche Prüfhinweise zu den Originalen

Diese Hinweise gehören zur Vorbereitung, nicht in den gesprochenen Vortrag. Die Originaldateien bleiben unverändert. Die BMS-Fokusansichten ergänzen ausschließlich Konturen im Dokument. Der Abgleich hat folgende Punkte ergeben, die vor der finalen Folienproduktion relevant sind.

Abbildung 7.12: RAM-Zahl nicht konsistent. Der quantisierte SOC-Balken zeigt etwa 3,5 KB. Tabelle 7.1 nennt 3,96 KB. Deshalb wird diese einzelne RAM-Zahl in der Erzählung nicht als feststehender Wert wiederholt. Vor der Übernahme des RAM-Panels müssen Grafik und Tabelle mit dem zugehörigen Firmware-Ergebnis abgeglichen werden.

Abbildung 7.12: Parameterzahl und Speicherbasis trennen. Das obere rechte Panel ist eine idealisierte Gewichtsspeicherung und setzt beim INT8-Balken ungefähr ein Viertel des gesamten FP32-Gewichtsspeichers an. Die real untersuchte Firmware quantisiert dagegen nur die rekurrenten Matrizen. Das untere linke Panel zeigt ihren Flash-Bedarf. Die oberen Parameterzahlen sind zudem nicht ohne Weiteres aus den aktuellen Dimensionen in Abschnitt 7.3 rekonstruierbar. Sie sollten vor einer Formelherleitung am konkreten Export geprüft werden. Die Erzählung verwendet die im Methodenteil explizit angegebenen MAC-Zahlen und keine aus dem Balken abgelesene Parameterherleitung.

Abbildung 7.8: kleine Zahlenabweichung. Die Originallegende nennt 2,33 und 2,78 Prozentpunkte für Pruned und Quantized. Der laufende Dissertationstext nennt 2,34 und 2,79. Die Erzählung verwendet deshalb für diesen Vergleich transparent gerundete Werte von 2,3 und 2,8. Die Abweichung von jeweils 0,01 Prozentpunkten bleibt ein abzugleichender Quellenpunkt, kein nachgewiesener anderer Effekt.

Abbildung 7.13: Messgrenze korrekt. Gezeigt wird die Host-Gesamtlatenz einschließlich UART. Die Kernelzeiten stammen aus Tabellen 7.1 und 7.2. Beide werden im Vortrag ausdrücklich getrennt. Die Zeitwerte charakterisieren die verwendeten Referenzkernels und Compilerbedingungen, nicht eine allgemeine Grenze für INT8-Inferenz auf Mikrocontrollern.

Korrelation und Robustheitsmechanismen. 5.1 zeigt lineare Korrelationen mit kumulierten Ladungsmengen, nicht eine kausale Stromwirkung oder gemessene Feature Importance. 6.5 zeigt Gain, nicht den additiven Offset. 6.9 betrifft definierte Erholung relativ zum ungestörten Vergleich, nicht garantierte absolute SOC-Genauigkeit. 6.13 zeigt einen Spannungsimpuls. Stromintegration und geringe Spannungsempfindlichkeit haben deshalb unterschiedliche Erklärungen. Der breitere Kontext des DD ist eine begründete strukturelle Interpretation, kein separater Ablationsnachweis.

Verschiedene Untersuchungen nicht gleichsetzen. Kapitel 6 verwendet für die Robustheit einen GRU-SOC-Zweig im Fensterbetrieb und einen gemeinsamen stündlichen SOH-LSTM. Kapitel 7 untersucht andere, schrittweise arbeitende LSTM-SOC- und LSTM-SOH-Modelle. Ihre MAE-Werte liefern keinen unmittelbaren Architekturvergleich. Die eigene H755-BMS-Platine und die H753-Hardwarebenchmarks sind ebenfalls getrennte Teile der experimentellen Grundlage.

Scalability richtig einordnen. Eine Architektur an unterschiedliche Ressourcenbudgets anzupassen, ist ein Deployment-Aspekt. Pruning kann das unterstützen. Eine nachgewiesene Übertragung auf beliebige Controller, Zellchemien oder größere Packs folgt daraus jedoch nicht. Deshalb werden diese nicht als bereits experimentell belegte Skalierungsergebnisse dargestellt.

Umfang dieser Bearbeitung. Dissertation, Originalbilder und vorhandene PowerPoints wurden nicht geändert. Die vorige Erzählfassung v2 bleibt im Sicherungsordner erhalten:

`_build_erzaehlfassung/previous_v2`